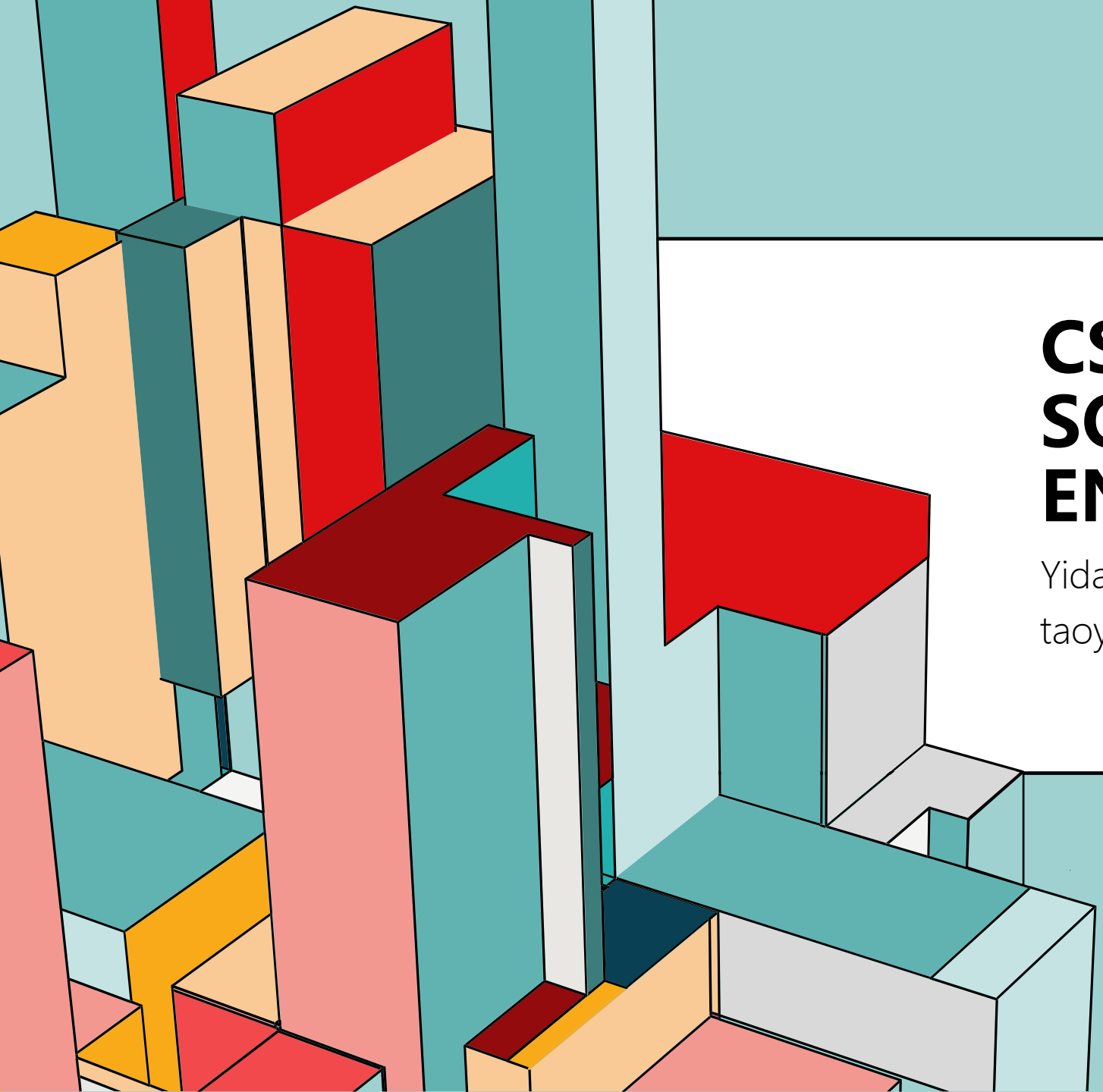
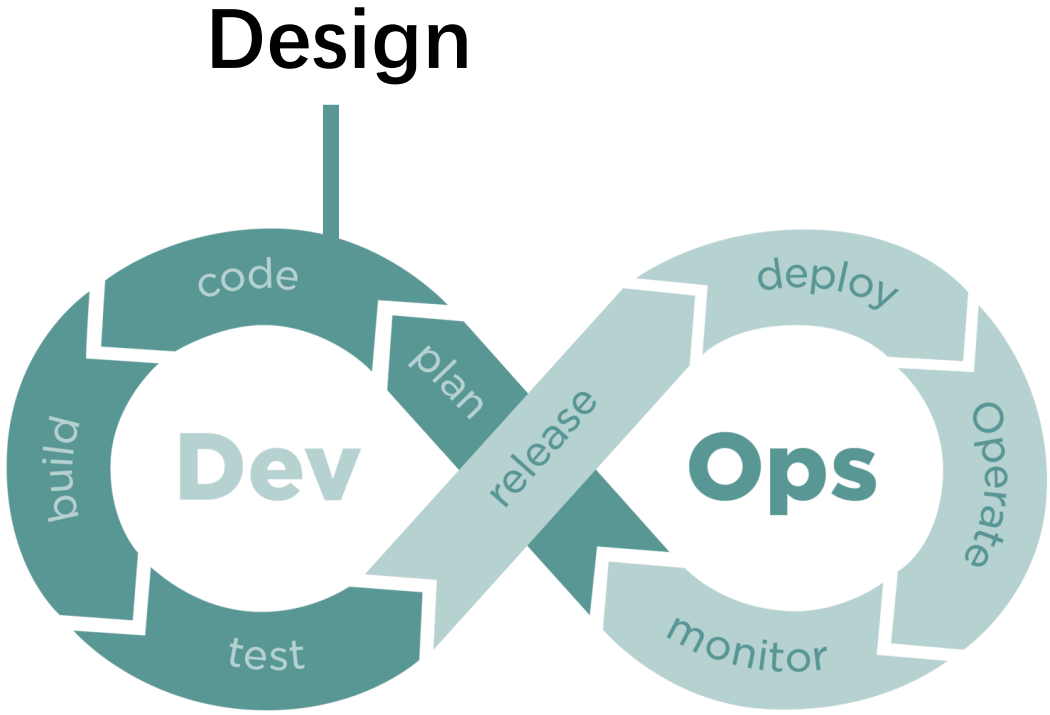


南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

CS304 SOFTWARE ENGINEERING

Yida Tao
taoyd@sustech.edu.cn

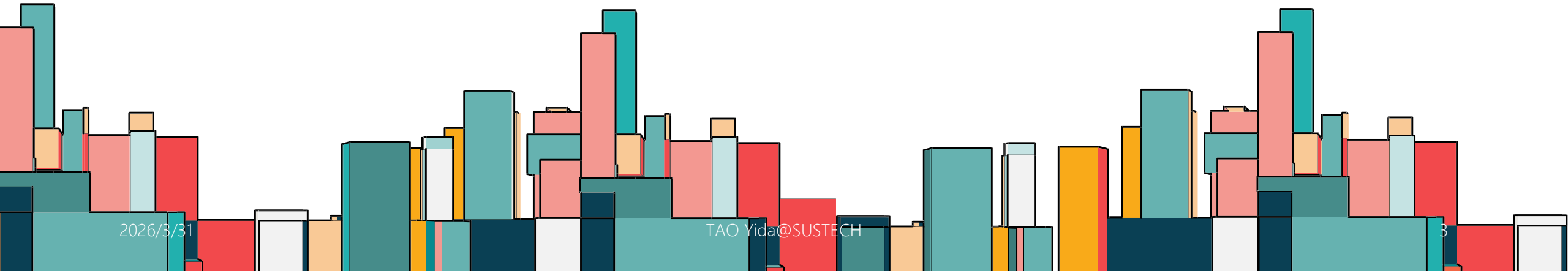
WHERE ARE WE NOW?

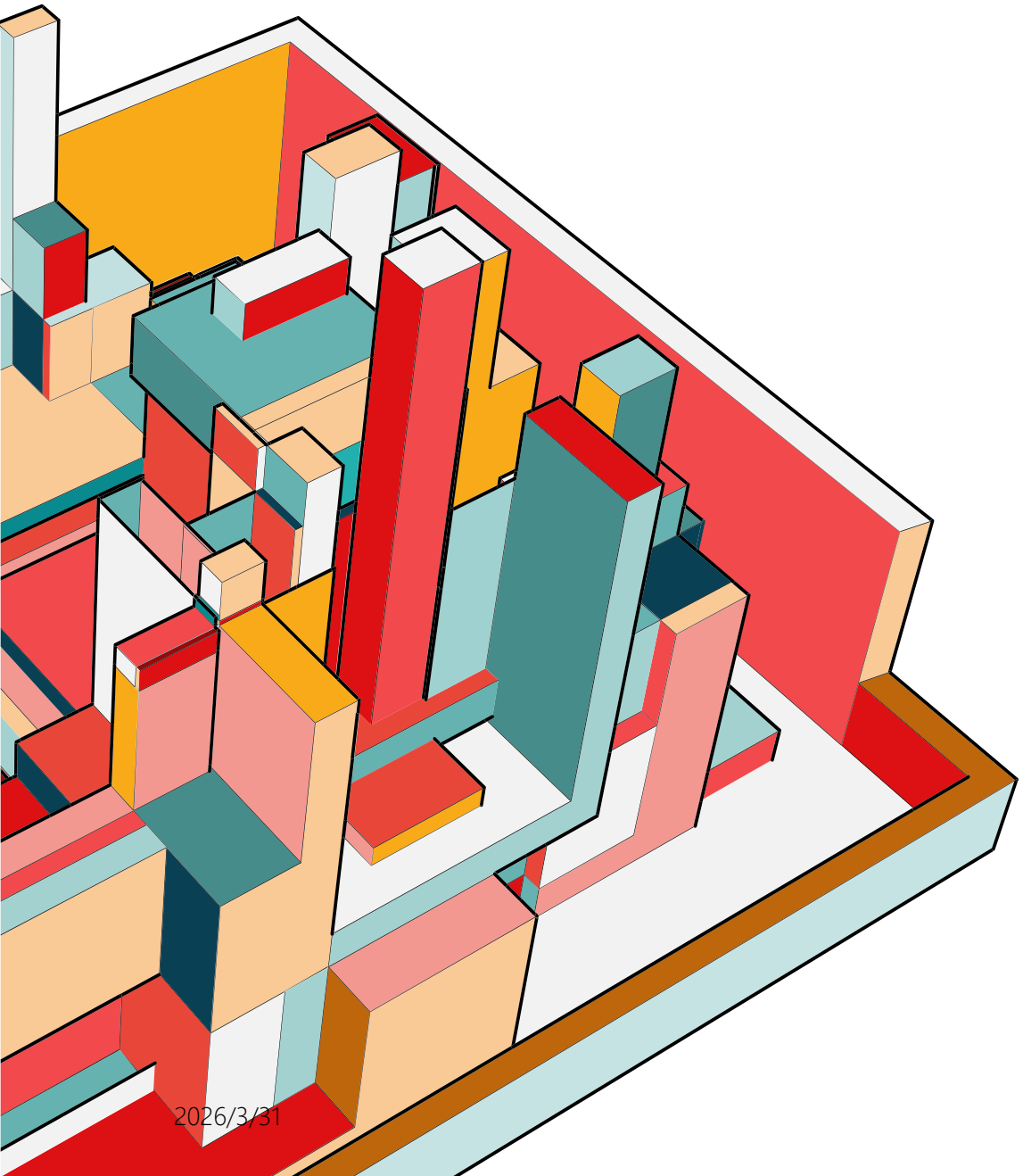


WHAT IS DESIGN?

It’s where you stand with a foot in two worlds—the world of technology and the world of people and human purposes—and you try to bring the two together.

- Mitch Kapor, “software design manifesto”





南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

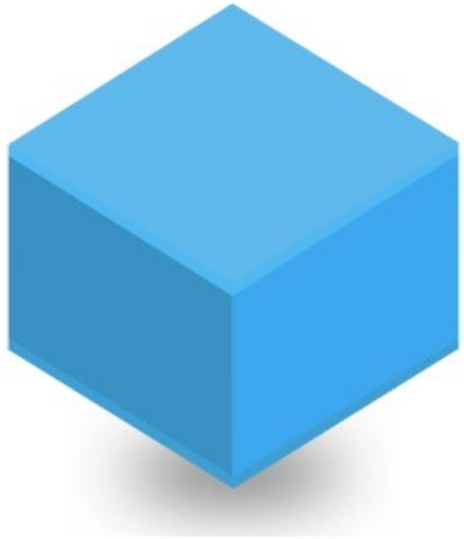
SOFTWARE DESIGN

- **Architectural design**
- **API design**
- **User interface design**
- Data design

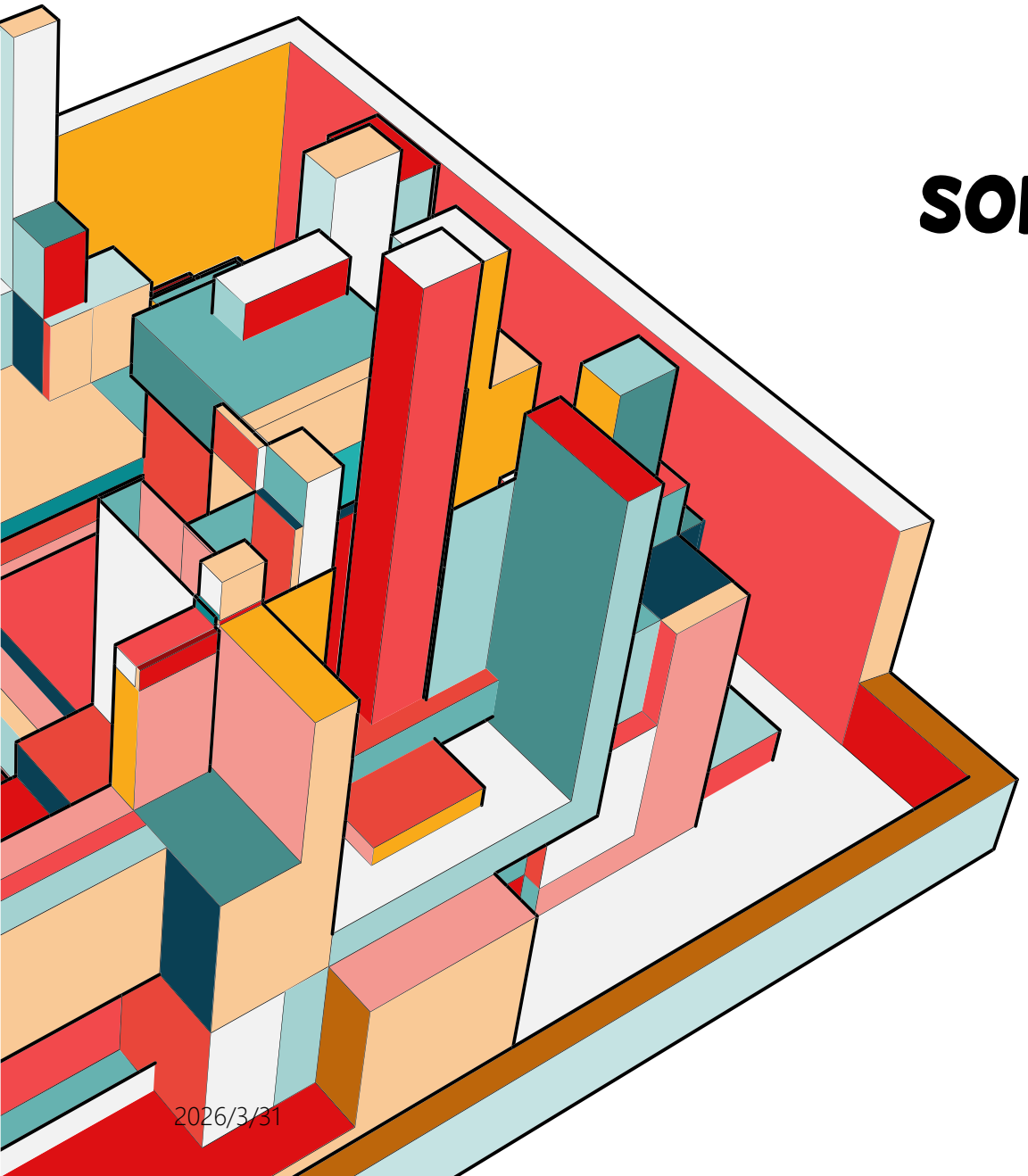
ARCHITECTURAL STYLE

SOFTWARE ARCHITECTURAL STYLE

Classic, Monolithic



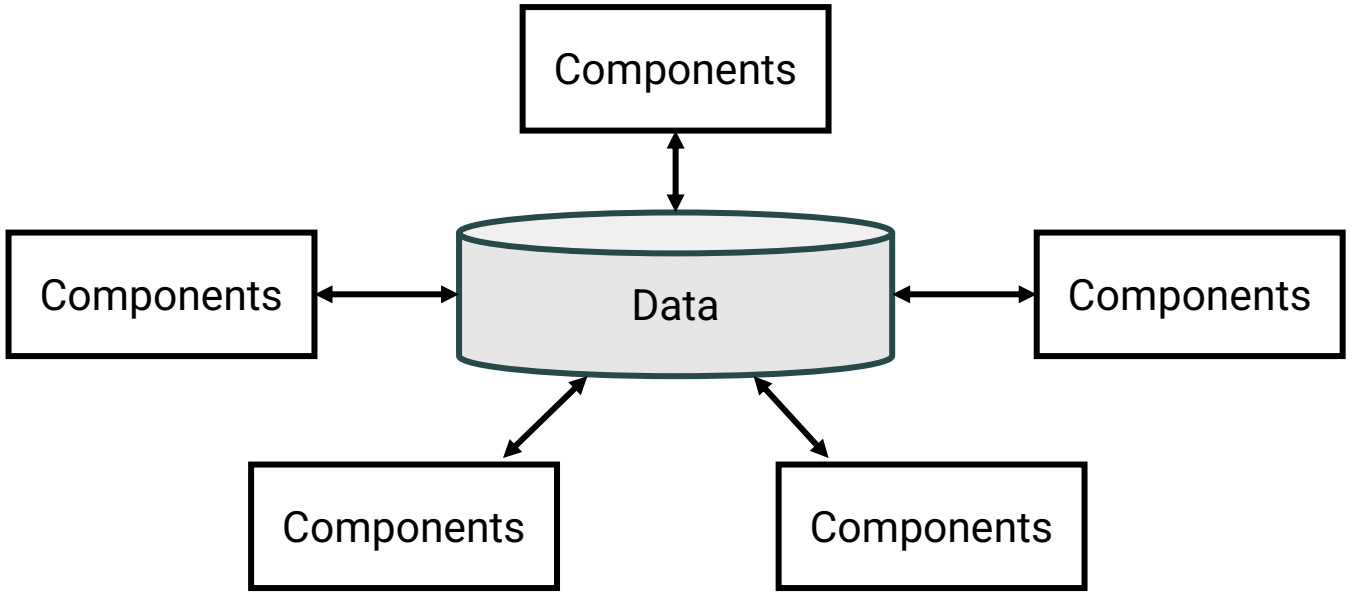
Service-based, Distributed, DevOps



SOFTWARE ARCHITECTURAL STYLE

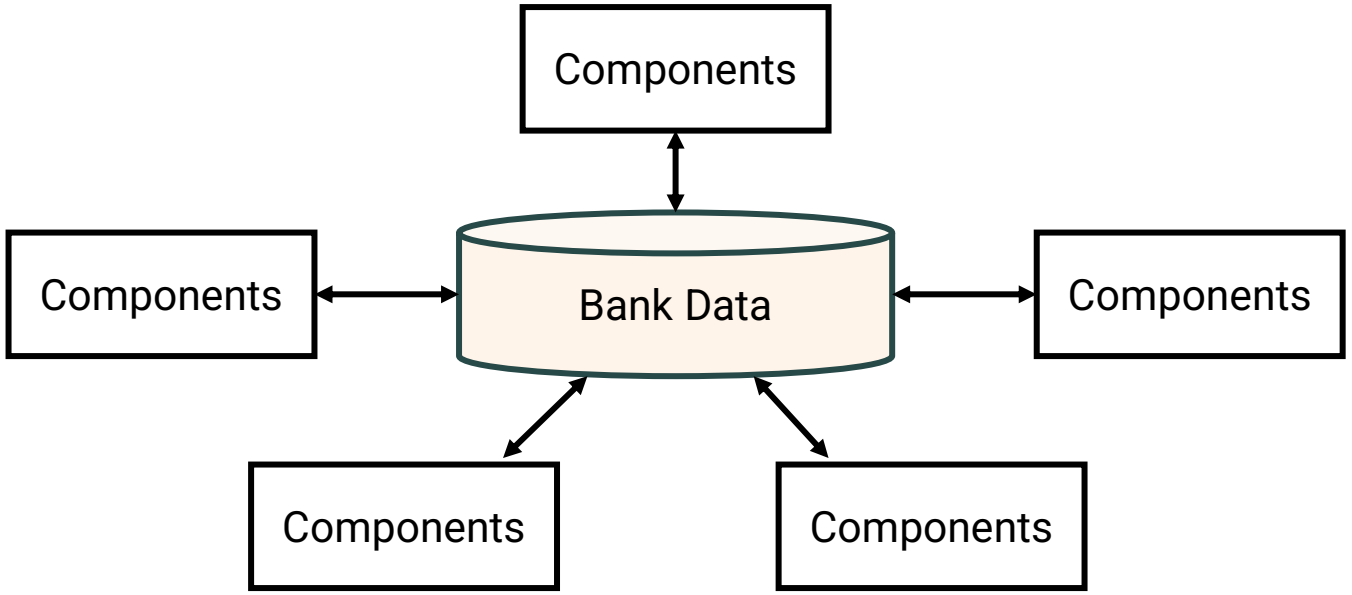
- Classic, Monolithic
  - Data-centered architecture
  - Data-flow architecture
  - Call-and-return architecture
  - Object-oriented architecture
  - Layered architecture
- Service-based, Distributed, DevOps
  - Microkernel architecture
  - Event-driven architecture
  - Microservice architecture

DATA-CENTERED ARCHITECTURES



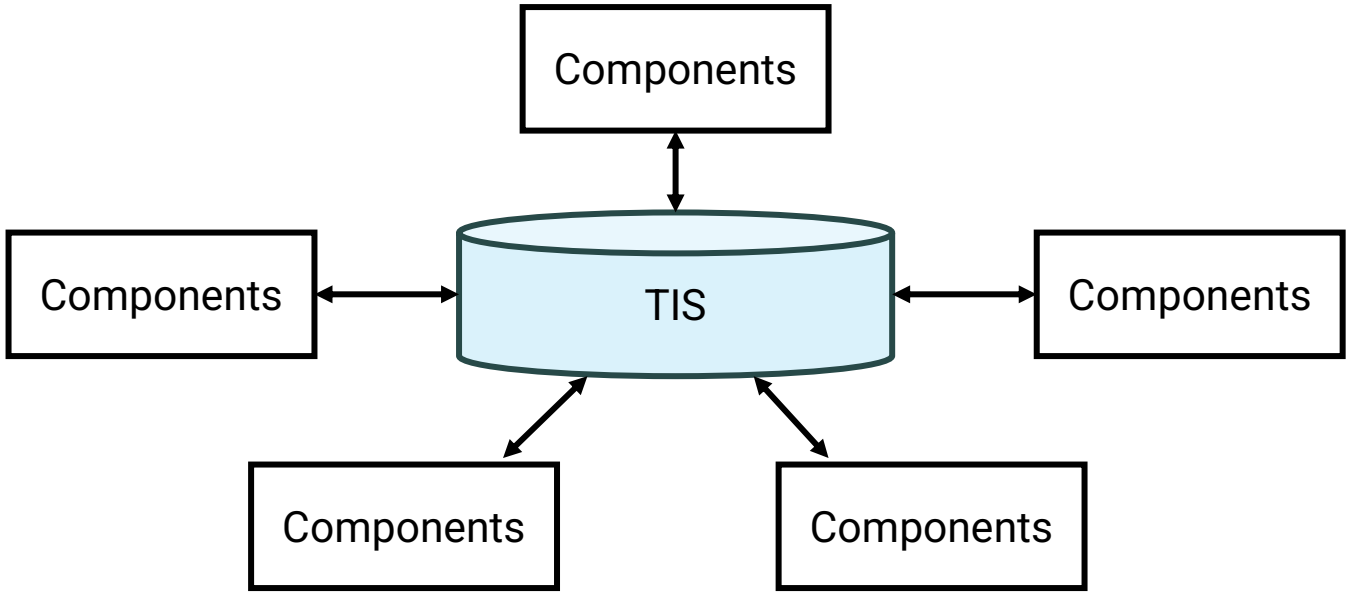
- A data store (e.g., database) resides at the center of this architecture
- The data store is accessed frequently by other components that update, add, delete, or modify data within the store

DATA-CENTERED ARCHITECTURES



- A data store (e.g., database) resides at the center of this architecture
- The data store is accessed frequently by other components that update, add, delete, or modify data within the store

DATA-CENTERED ARCHITECTURES

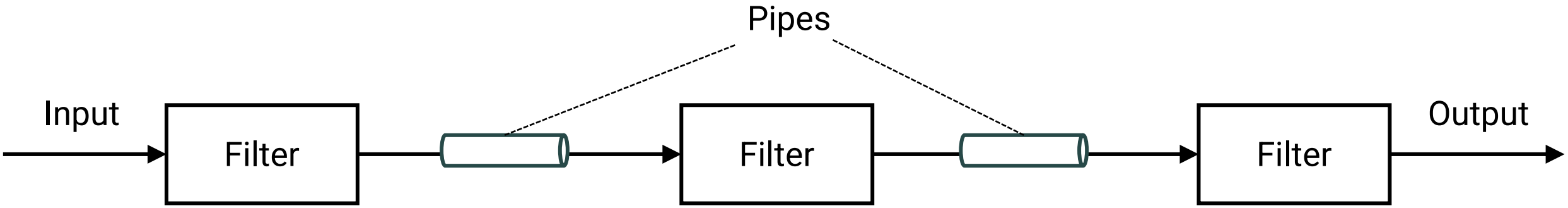


Good: single source of truth

- A data store (e.g., database) resides at the center of this architecture
- The data store is accessed frequently by other components that update, add, delete, or modify data within the store

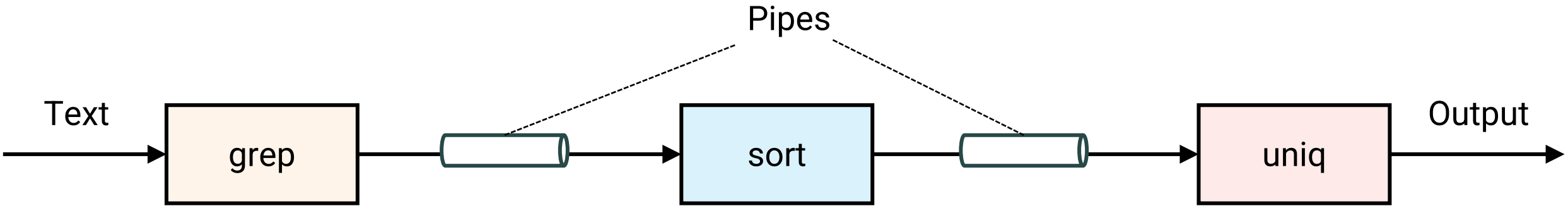
Bad: single point of failure

DATA-FLOW ARCHITECTURES



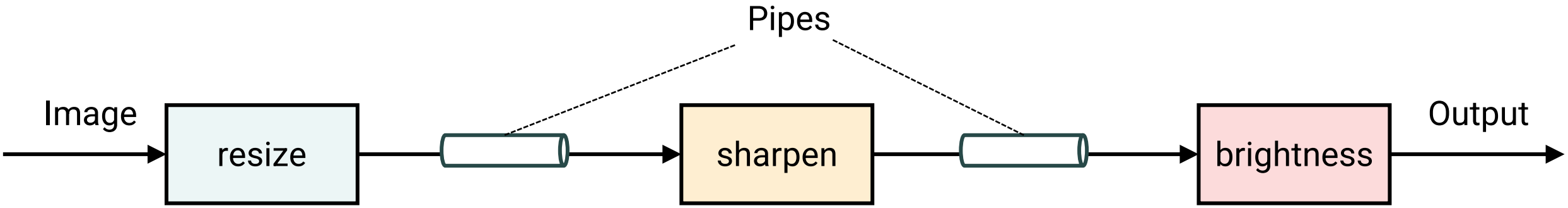
- Input data transformed step by step into output
- Also called the **Pipe-and-filter** pattern
- Each filter is independent, with no knowledge of upstream or downstream filters

DATA-FLOW ARCHITECTURES



- Input data transformed step by step into output
- Also called the Pipe-and-filter pattern
- Each filter is independent, with no knowledge of upstream or downstream filters

DATA-FLOW ARCHITECTURES

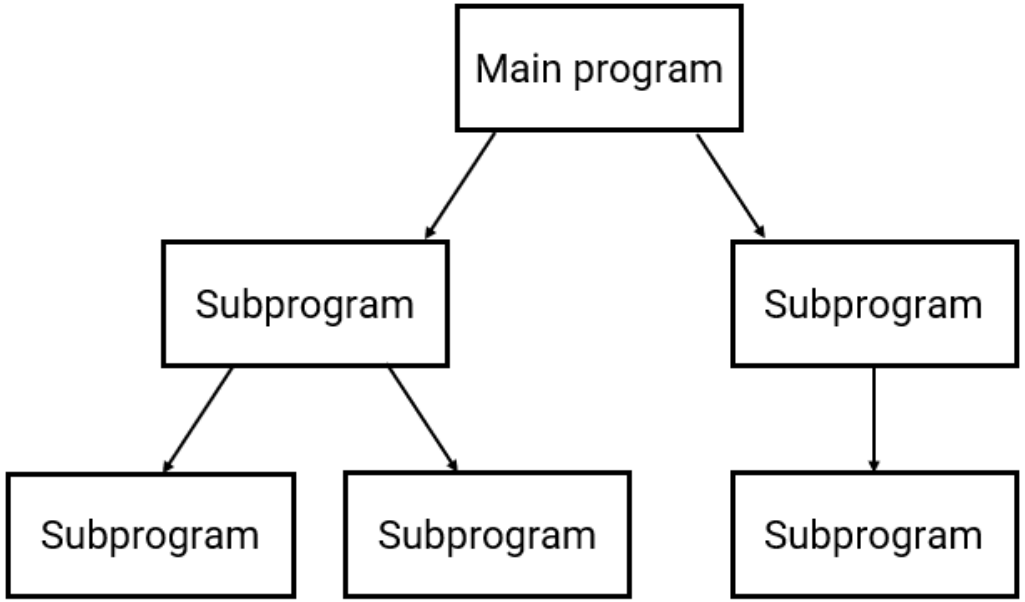


- Input data transformed step by step into output
- Also called the Pipe-and-filter pattern
- Each filter is independent, with no knowledge of upstream or downstream filters

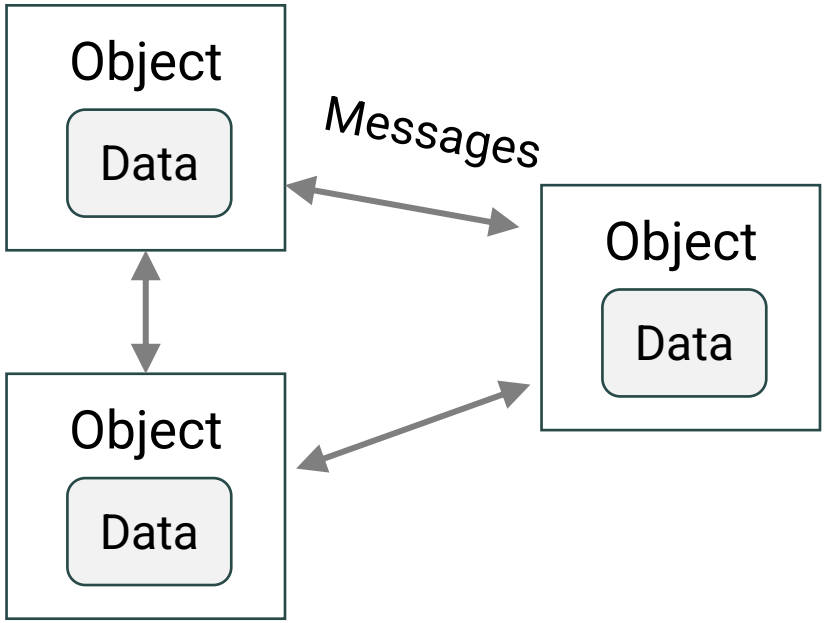
Good: clarity and modularity

Bad: bottlenecks, non-interactive

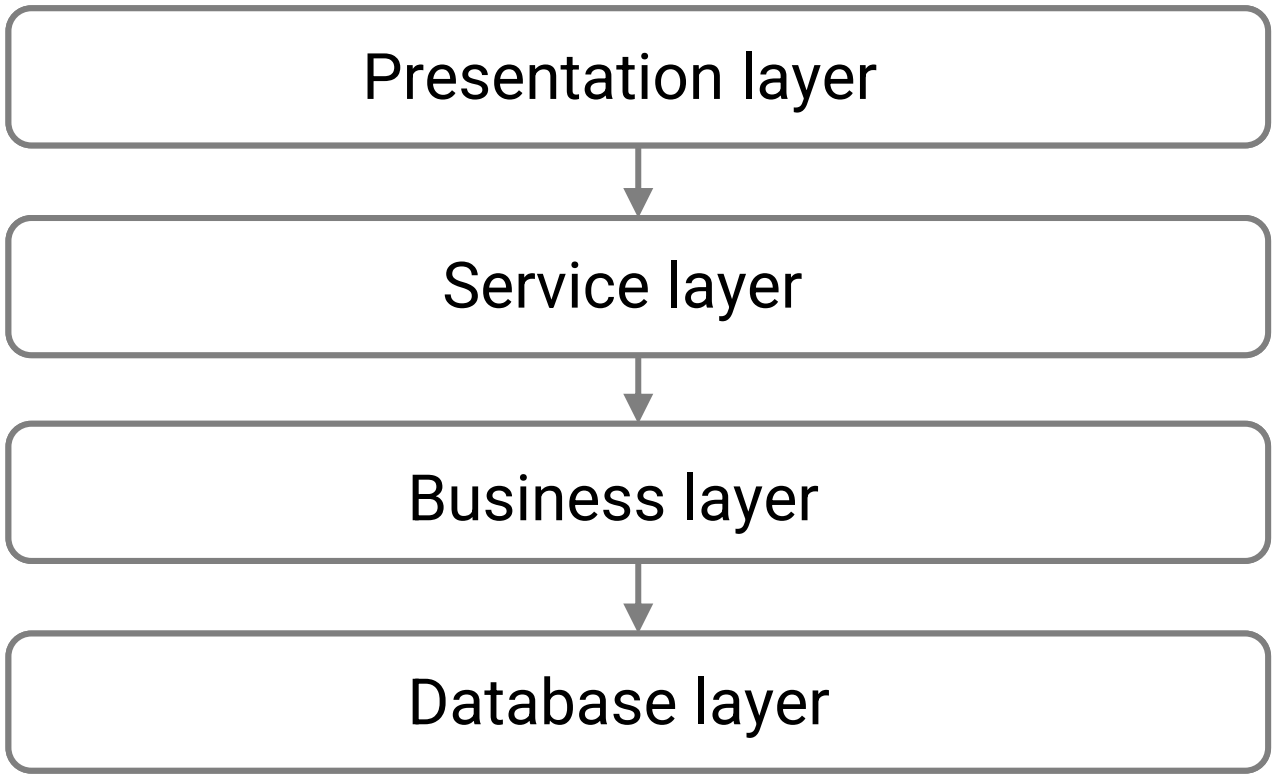
CALL AND RETURN ARCHITECTURES



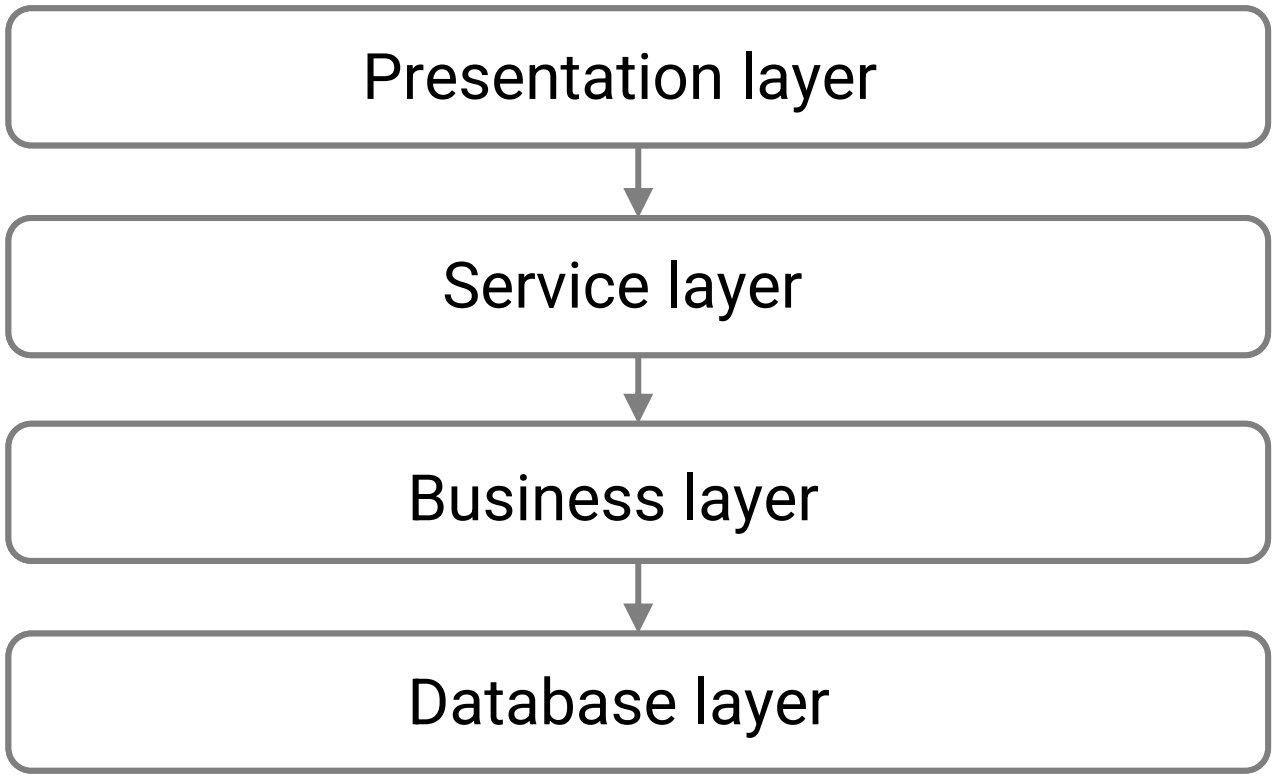
OBJECT-ORIENTED ARCHITECTURES



LAYERED ARCHITECTURES



LAYERED ARCHITECTURES



Course management system

User interaction: click “enroll”

Get and prepare the result

Enrollment rules, grade calculations

Raw data: students, courses, and grades

LAYERED ARCHITECTURES

应用架构设计基础

应用架构设计基础——三层架构

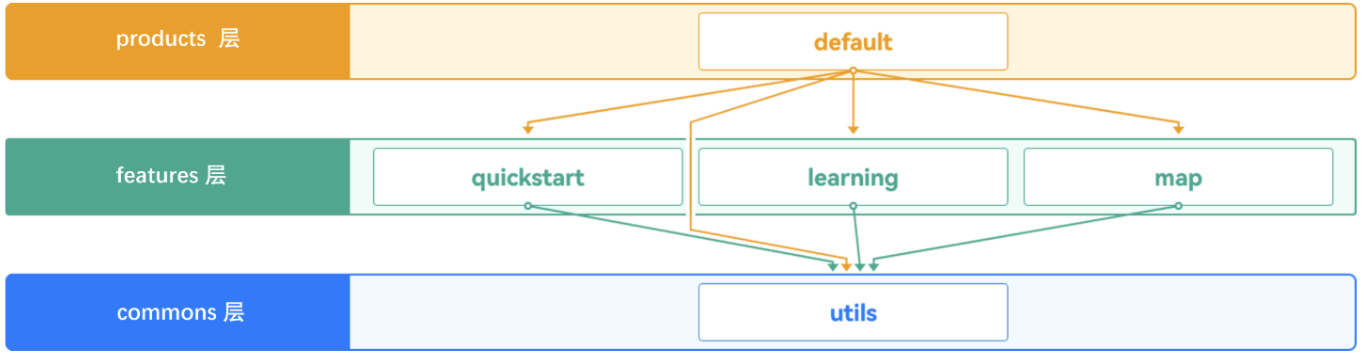
学习如何应用三层架构开发应用

更新时间 2025-11-12

预计学习时长 20分钟

HarmonyOS

https://developer.huawei.com/consumer/cn/codelabsPortal/carddetails/tutorials_Next-BasicArchitectureDesignPart2



三层工程结构如下：

- commons（公共能力层）：用于存放公共基础能力集合（如工具库、公共配置等）。commons层可编译成一个或多个HAR包或HSP包，只可以被products和features依赖，不可以反向依赖。
- features（基础特性层）：用于存放基础特性集合（如应用中相对独立的各个功能的UI及业务逻辑实现等）。各个feature高内聚、低耦合、可定制，供产品灵活部署。不需要单独部署的feature通常编译为HAR包或HSP包，供products或其它feature使用。需要单独部署的feature通常编译为Feature类型的HAP包，和products下Entry类型的HAP包进行组合部署。features层可以横向调用及依赖common层，同时可以被products层不同设备形态的HAP所依赖，但是不能反向依赖products层。
- products（产品定制层）：用于针对不同设备形态进行功能和特性集成。products层各个子目录各自编译为一个Entry类型的HAP包，作为应用主入口。products层不可以横向调用。

PROBLEMS WITH MONOLITHIC?

Netflix users suffering service's longest outage ever

An undisclosed error has taken all 55 Netflix distribution centers offline. Customer-facing site remains up.

Rafe Needleman

Aug. 15, 2008 11:04 a.m. PT

2 min read

Netflix, and its customers, are currently experiencing the worst system failure in the DVD shipper's history. The company shipped no discs on Tuesday, only "a few" yesterday, and so far none today. Affected customers are receiving e-mails telling them their discs are delayed.

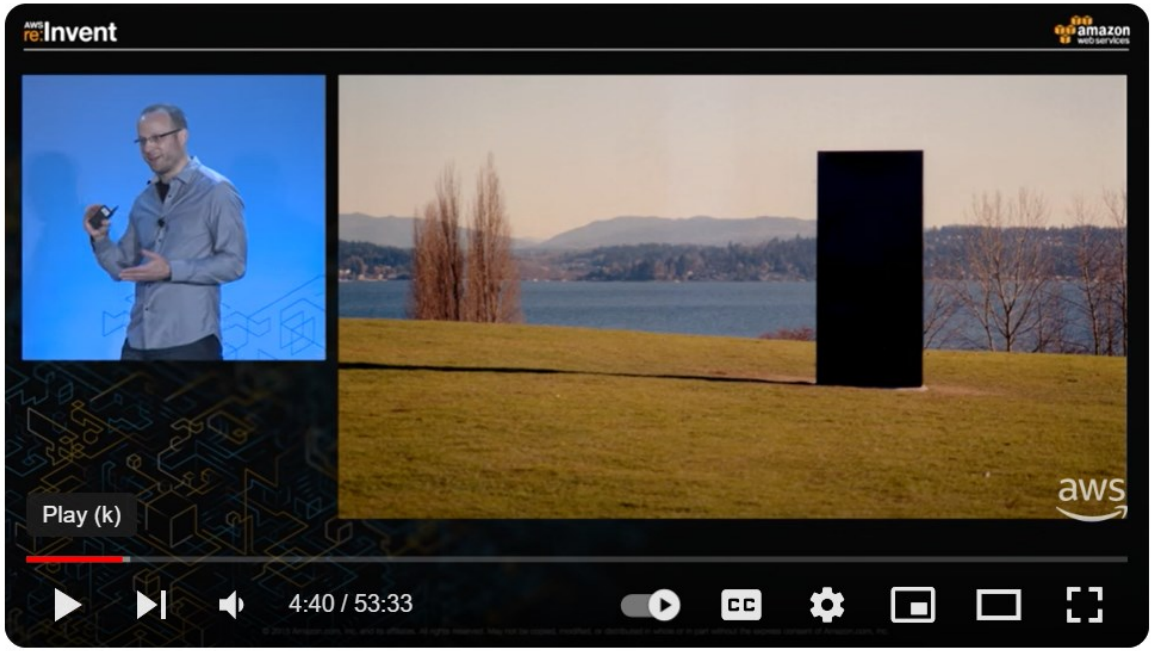
In 2009, Netflix faced growing pains as its **monolithic architecture** struggled to cope with the demand for video streaming services.

Determined to emerge victorious, Netflix migrated to a **cloud-based microservices architecture**, paving the way for its rapid rise.

PROBLEMS WITH MONOLITHIC?

PROBLEMS WITH MONOLITHIC?

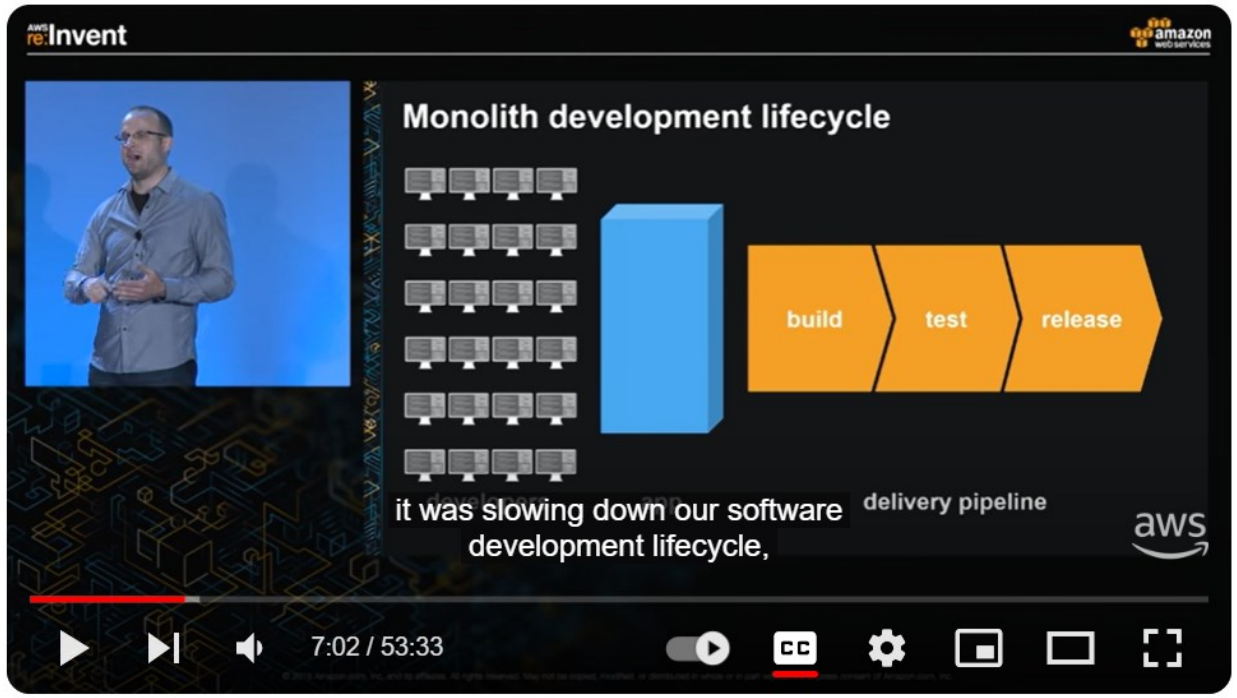
"If you go back to 2001," stated Amazon AWS senior manager for product management Rob Brigham, "the Amazon.com retail website was a large **architectural monolith**."



AWS re:Invent 2015: DevOps at Amazon: A Look at Our Tools and Processes (DVO202)

PROBLEMS WITH MONOLITHIC?

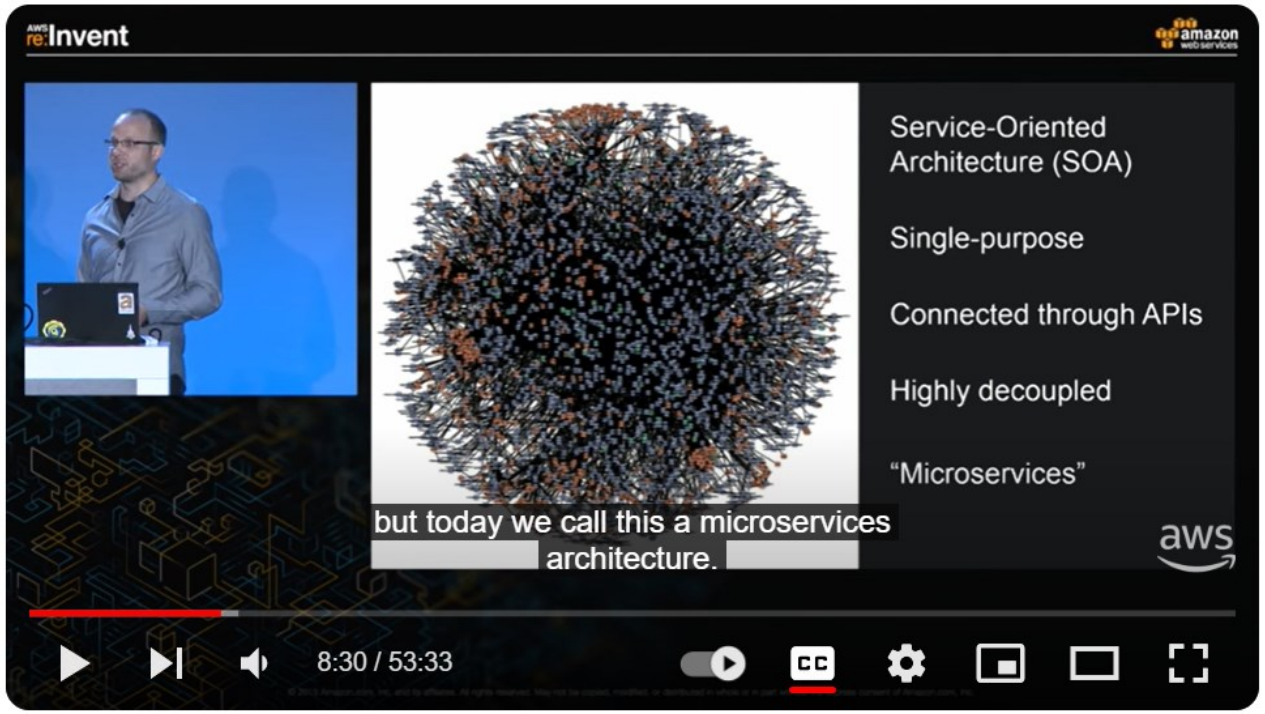
"Monolithic architecture adds large overhead to the process, frustrates developers, and slows down the entire software development lifecycle."



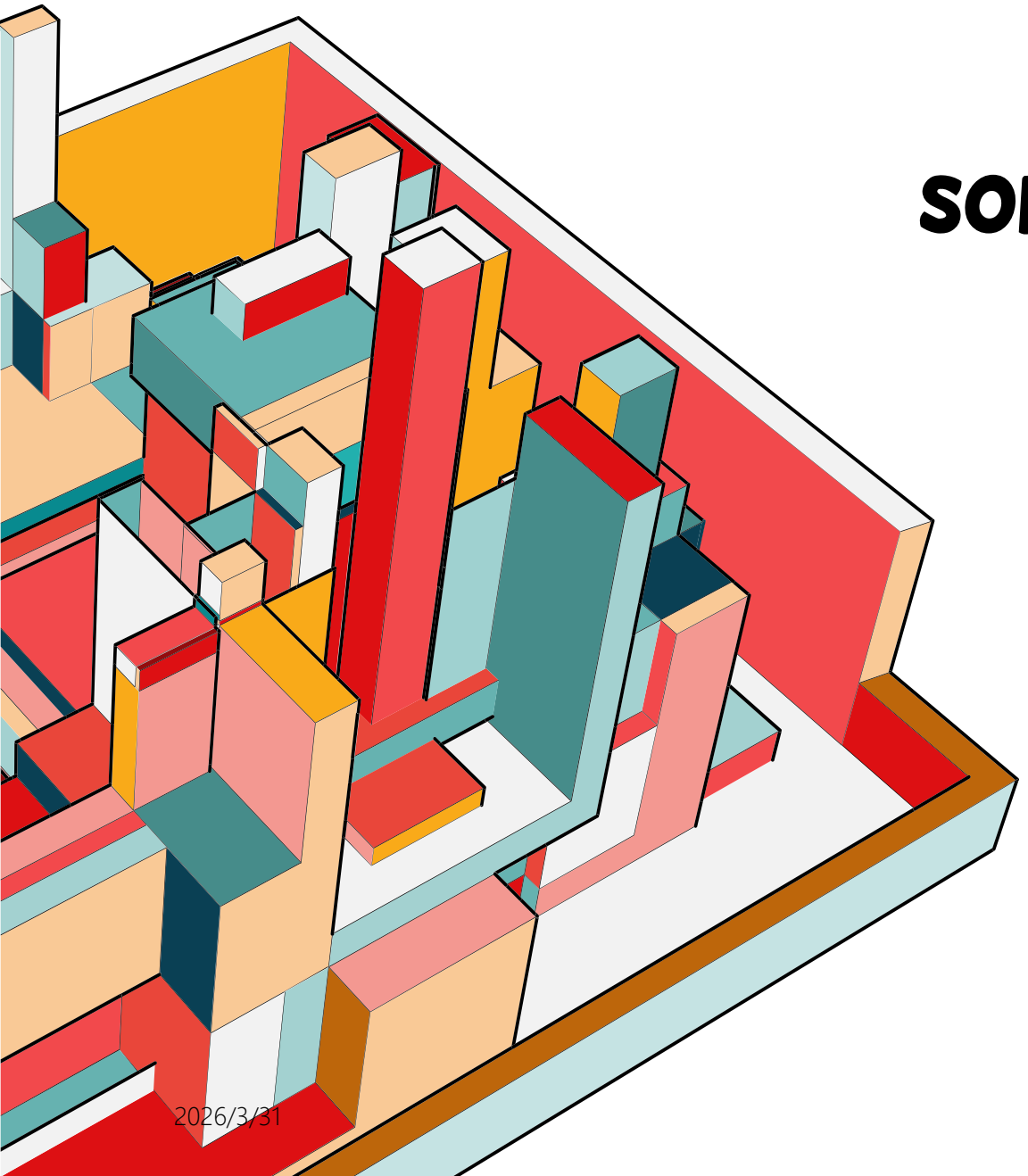
AWS re:Invent 2015: DevOps at Amazon: A Look at Our Tools and Processes (DVO202)

PROBLEMS WITH MONOLITHIC?

"We teased it apart into service-oriented architecture."



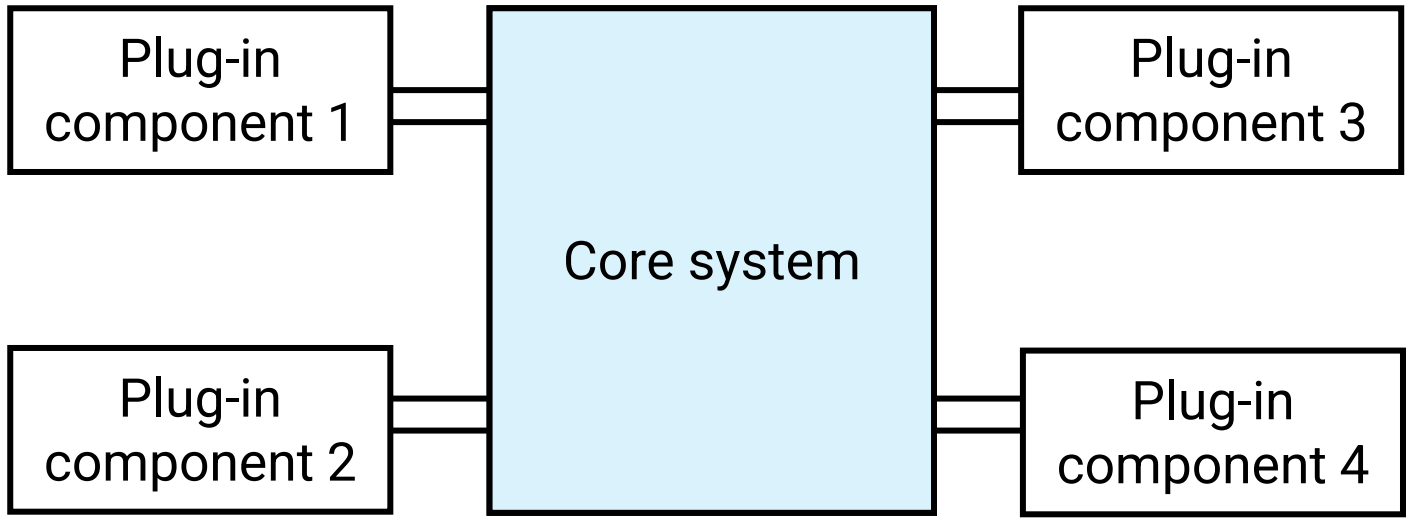
AWS re:Invent 2015: DevOps at Amazon: A Look at Our Tools and Processes (DVO202)



SOFTWARE ARCHITECTURAL STYLE

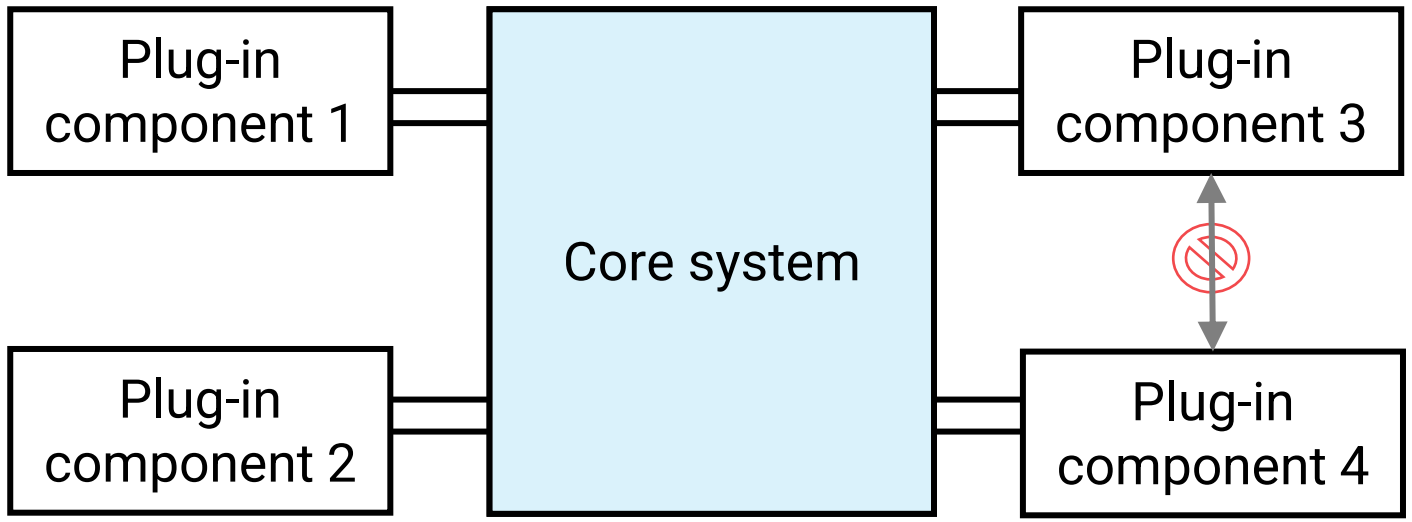
- Classic, Monolithic
  - Data-centered architecture
  - Data-flow architecture
  - Call-and-return architecture
  - Object-oriented architecture
  - Layered architecture
- Service-based, Distributed, DevOps
  - Microkernel architecture
  - Event-driven architecture
  - Microservice architecture

MICROKERNEL ARCHITECTURES



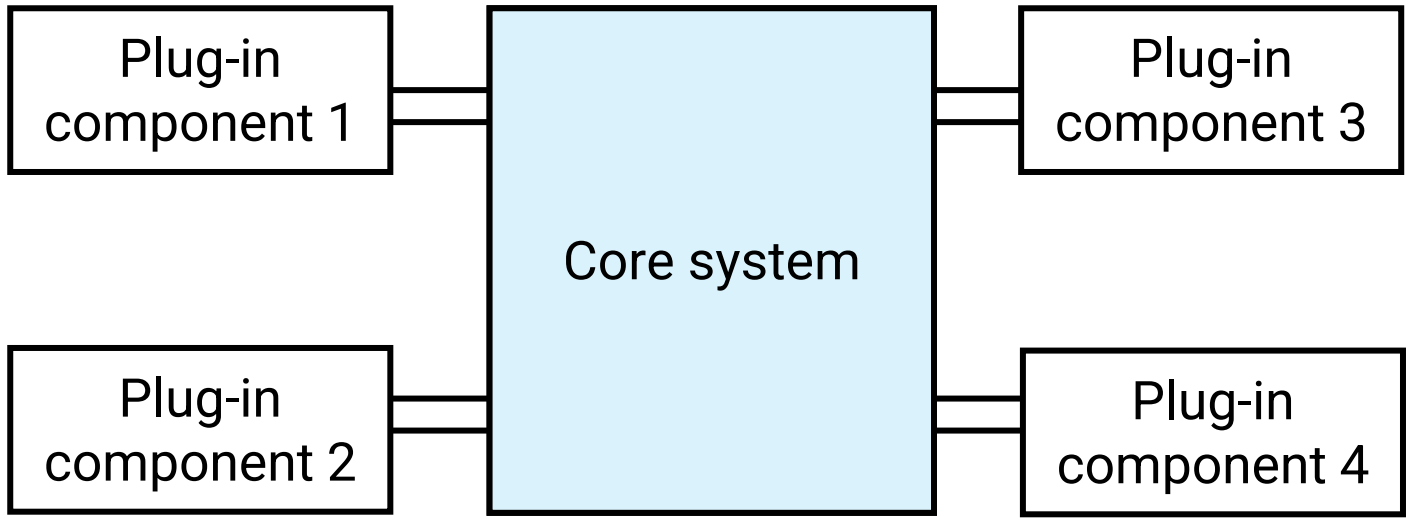
- Also known as the **plug-in architecture pattern**
- **Core system**: only the **minimal functionality** required to make the system operational

MICROKERNEL ARCHITECTURES



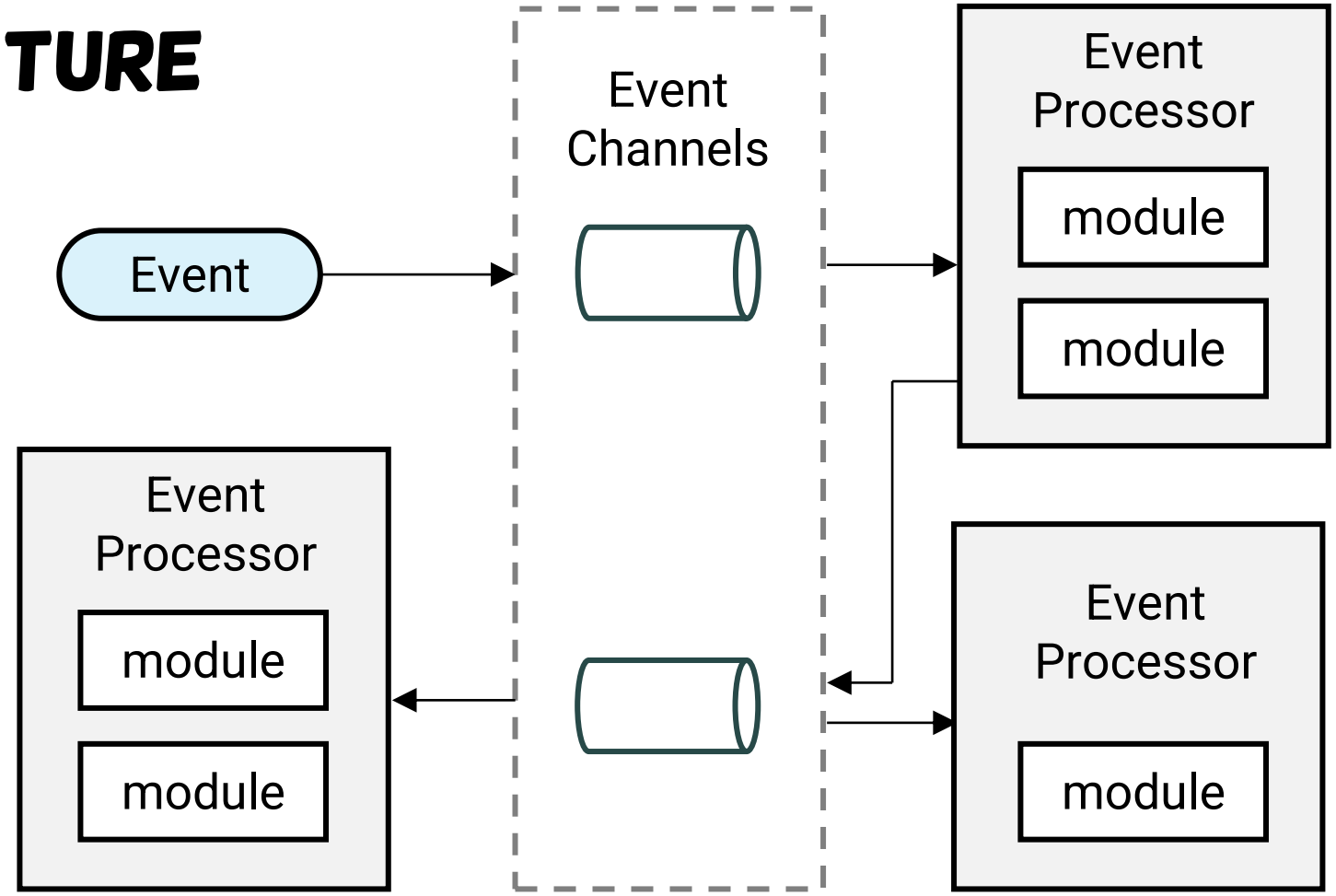
Plug-in component: stand-alone, independent components that contain specialized processing, additional features, and custom code that is meant to enhance or extend the core system to produce additional business capabilities.

MICROKERNEL ARCHITECTURES

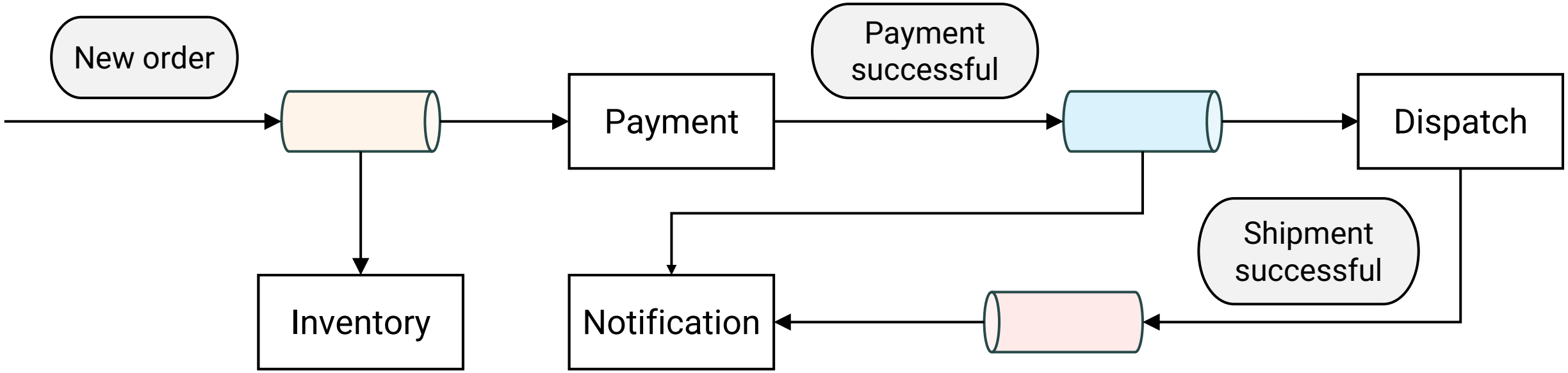


EVEN-DRIVEN ARCHITECTURE

- Core Components:
  - **Event Producer**: Initiates and generates events.
  - **Event Consumer/Processor**: Reacts to and processes events.
  - **Event channel/mediator/broker**: orchestration
- Asynchronous and distributed



EVENT-DRIVEN ARCHITECTURE



E-commerce application: placing a new order

MICROSERVICE ARCHITECTURE

The microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API.

These services are built around business capabilities and independently deployable by fully automated deployment machinery.

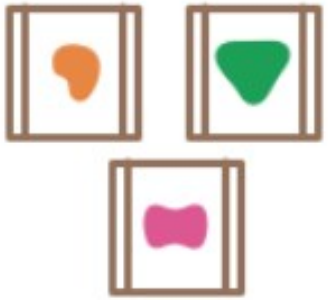
https://martinfowler.com/articles/microservices.html

MICROSERVICE ARCHITECTURE

Services

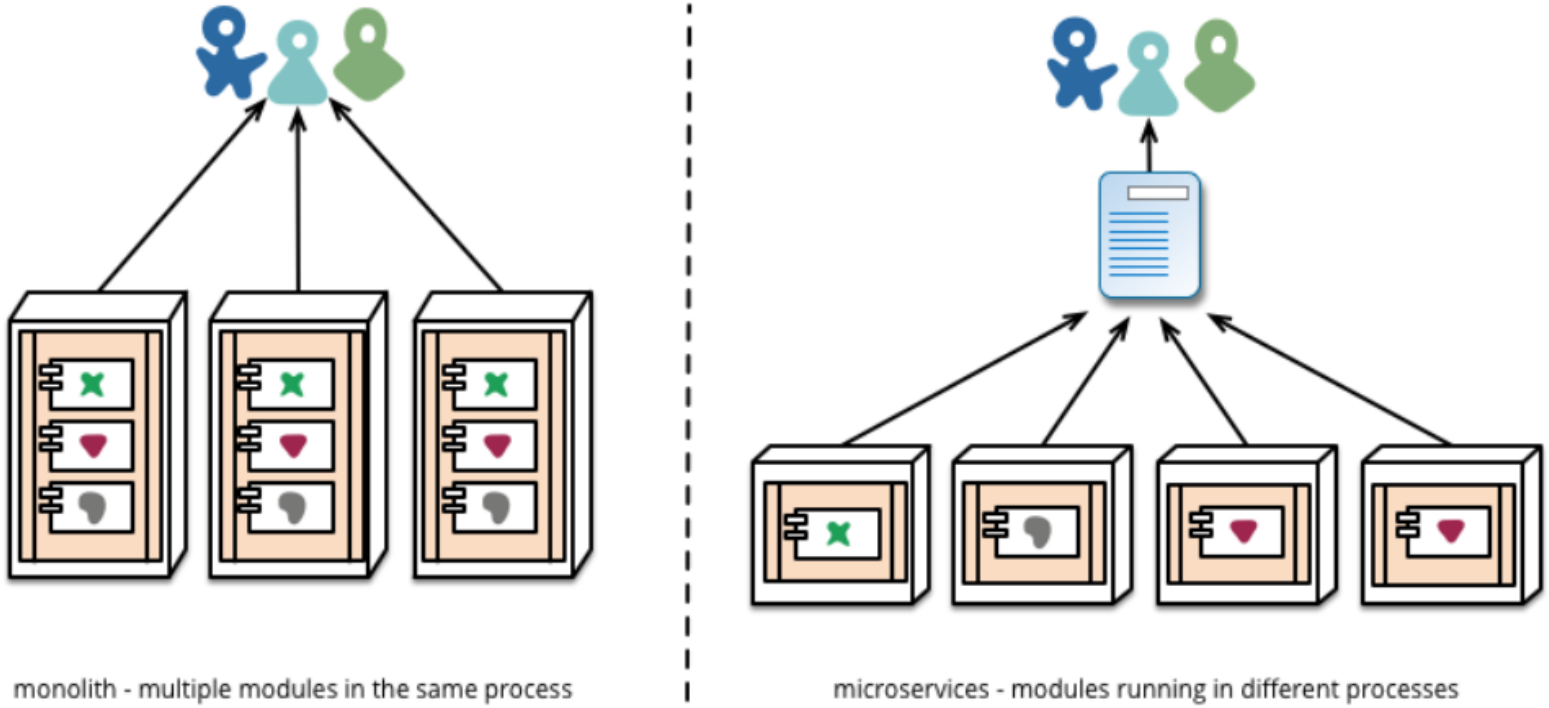
A monolithic application puts all its functionality into a single process...

A microservices architecture puts each element of functionality into a separate service...



https://martinfowler.com/articles/microservices.html

Deployment



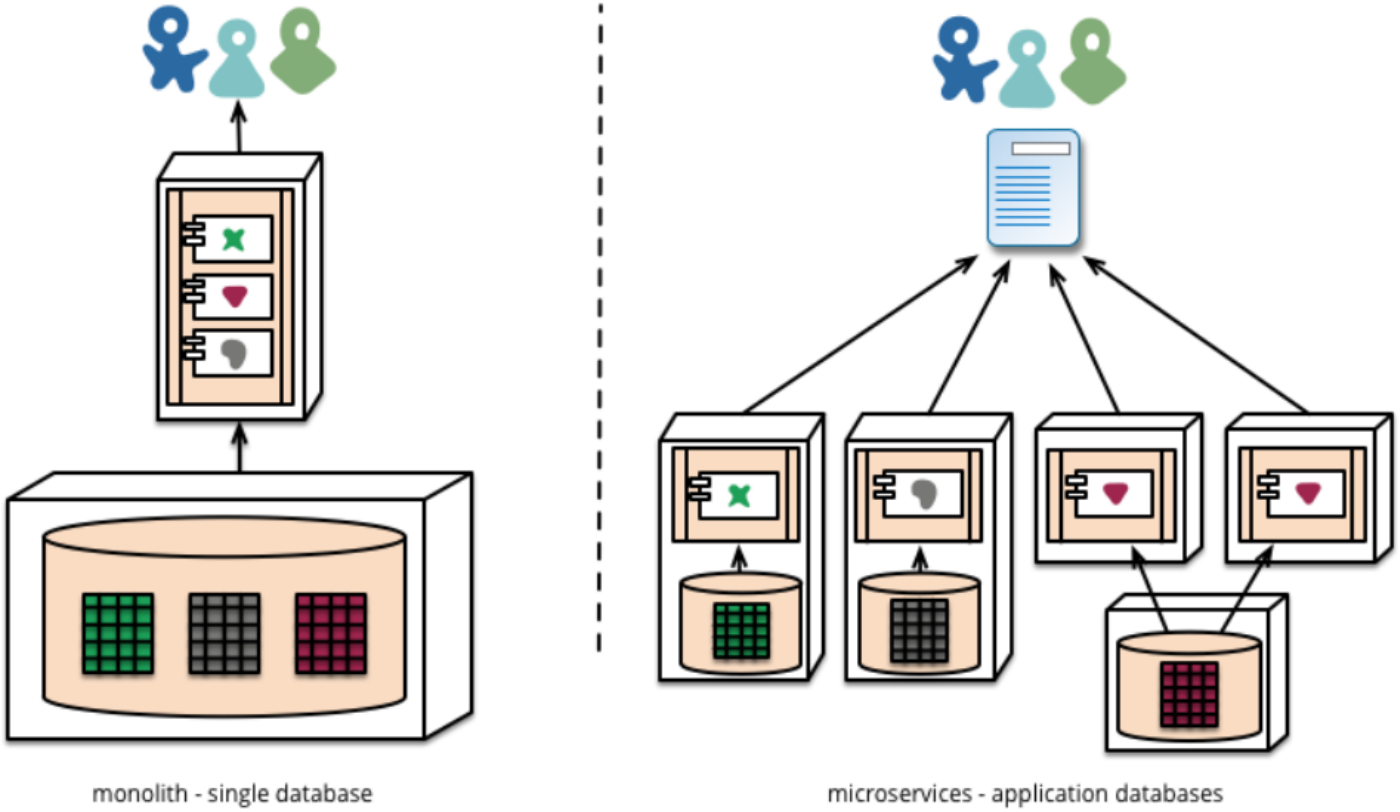
monolith - multiple modules in the same process

microservices - modules running in different processes

MICROSERVICE ARCHITECTURE

https://martinfowler.com/articles/microservices.html

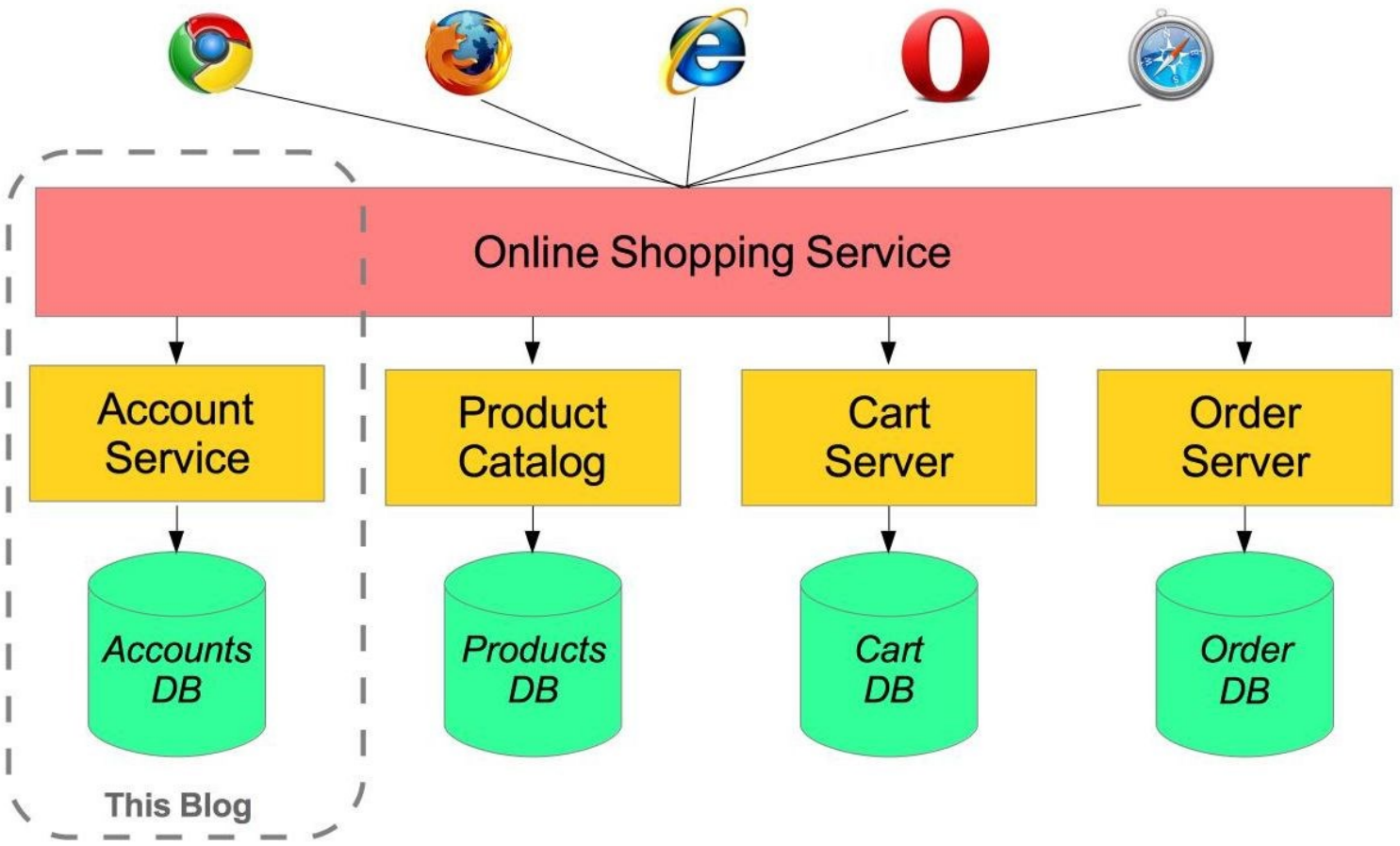
Decentralized Data Management



MICROSERVICE ARCHITECTURE

https://martinfowler.com/articles/microservices.html

Example: online shopping



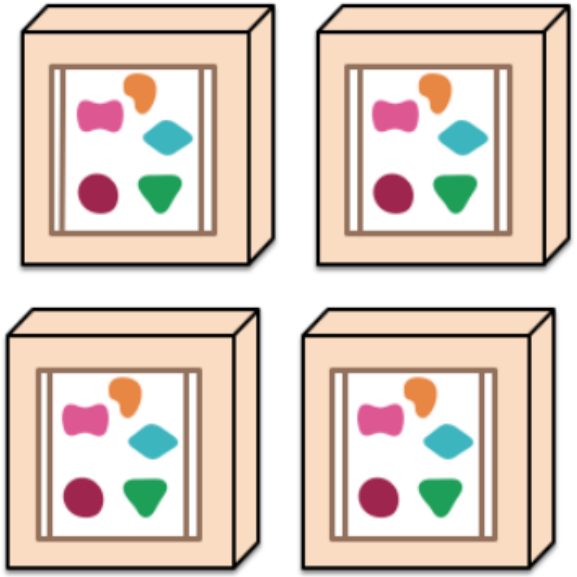
https://spring.io/blog/2015/07/14/microservices-with-spring

MICROSERVICE ARCHITECTURE

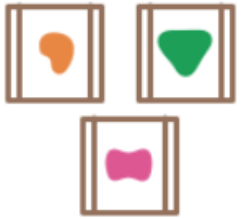
Scale

A monolithic application puts all its functionality into a single process...

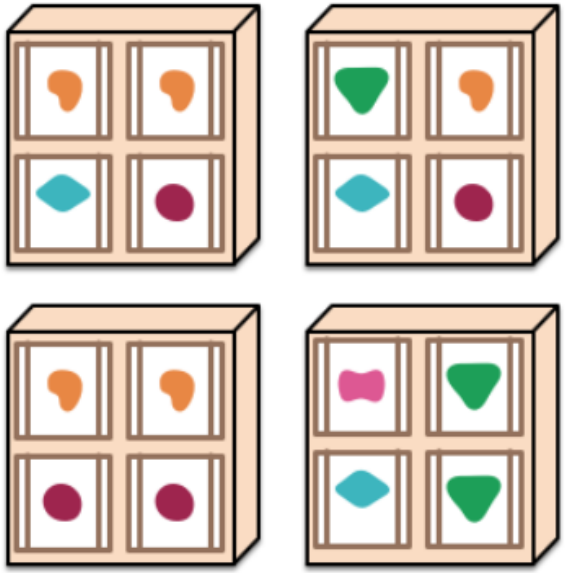
... and scales by replicating the monolith on multiple servers



A microservices architecture puts each element of functionality into a separate service...



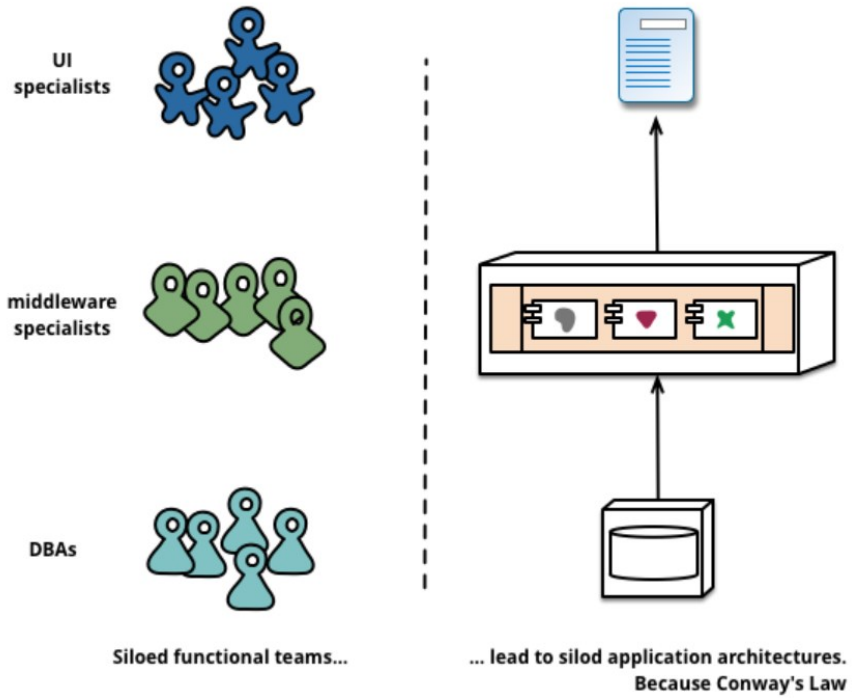
... and scales by distributing these services across servers, replicating as needed.



MICROSERVICE ARCHITECTURE

https://martinfowler.com/articles/microservices.html

Team organization

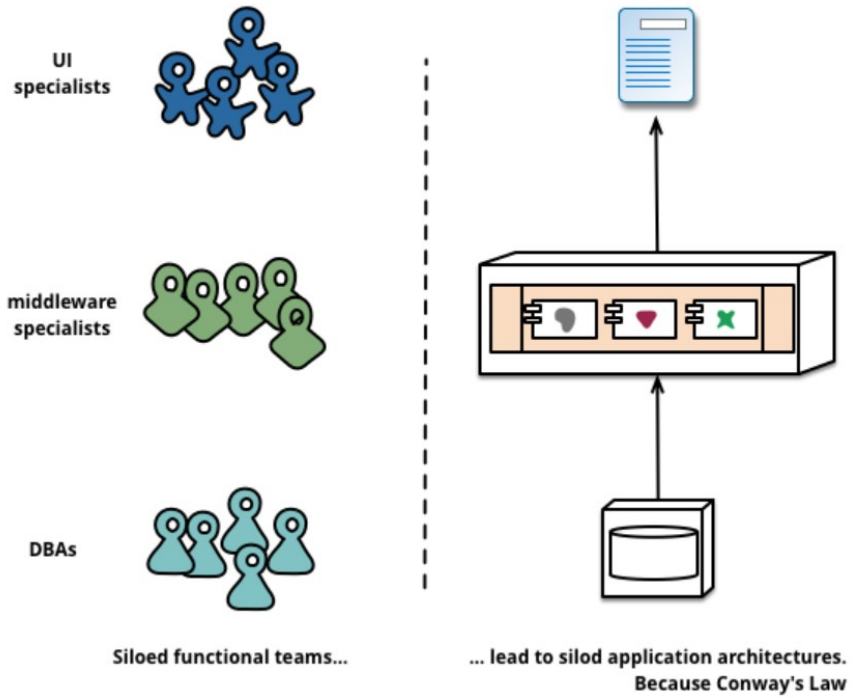


Monolithic (layered)

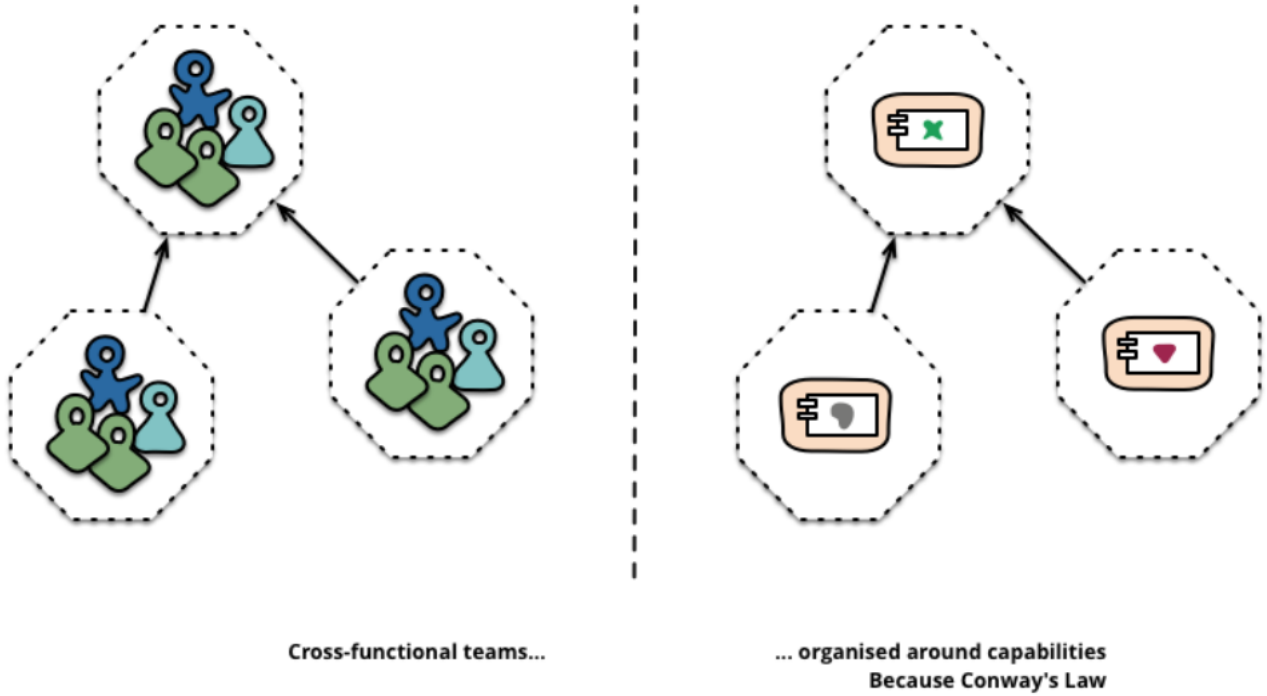
Conway’s Law
“Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure.”

https://martinfowler.com/articles/microservices.html

Team organization

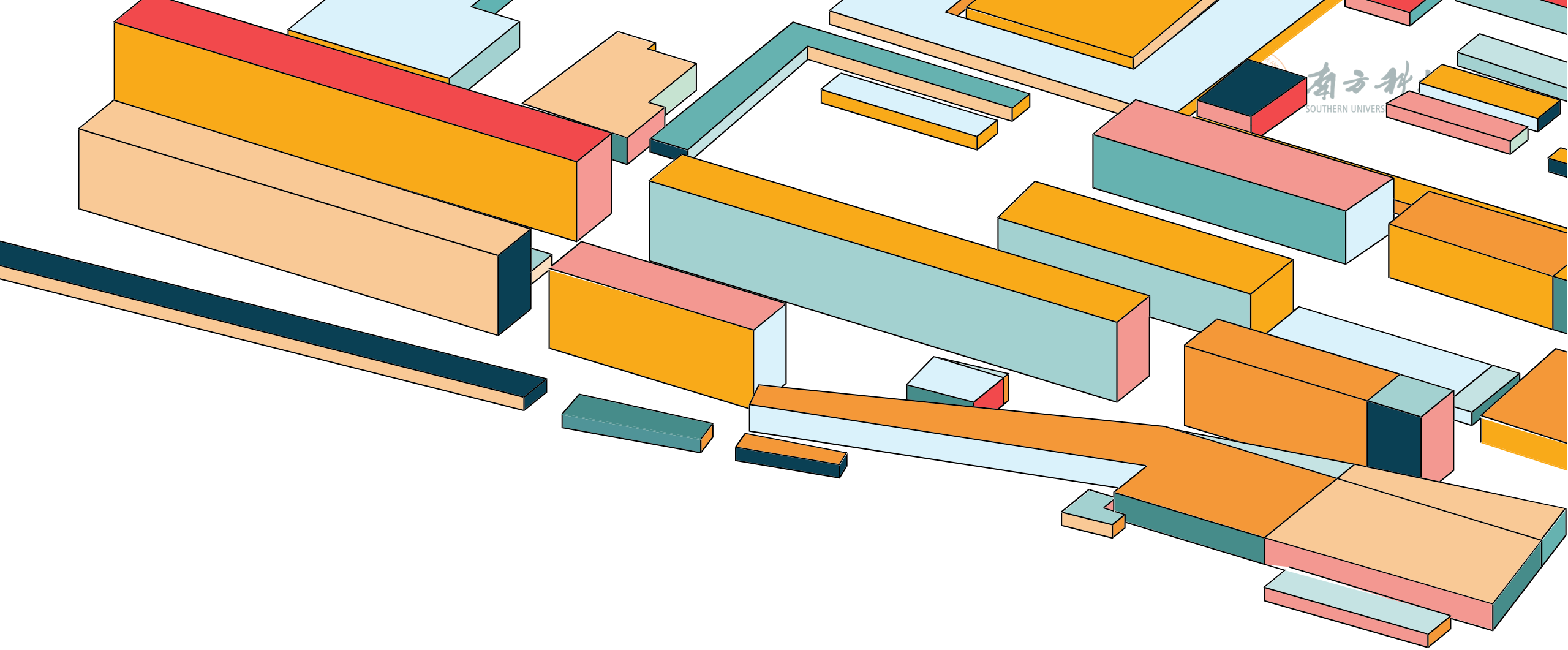


Monolithic (layered)



Microservice

https://martinfowler.com/articles/microservices.html



API DESIGN

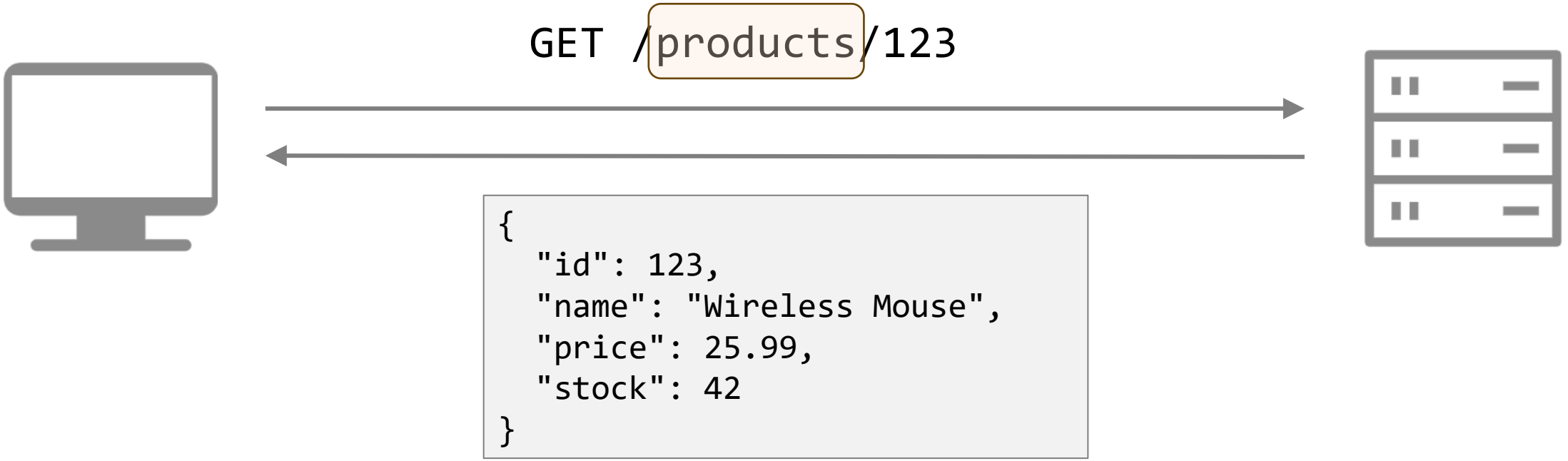
SERVICE COMMUNICATION STYLES

- Synchronous
  - The client expects a timely response from the service and block while it waits.
  - RESTful API, gRPC

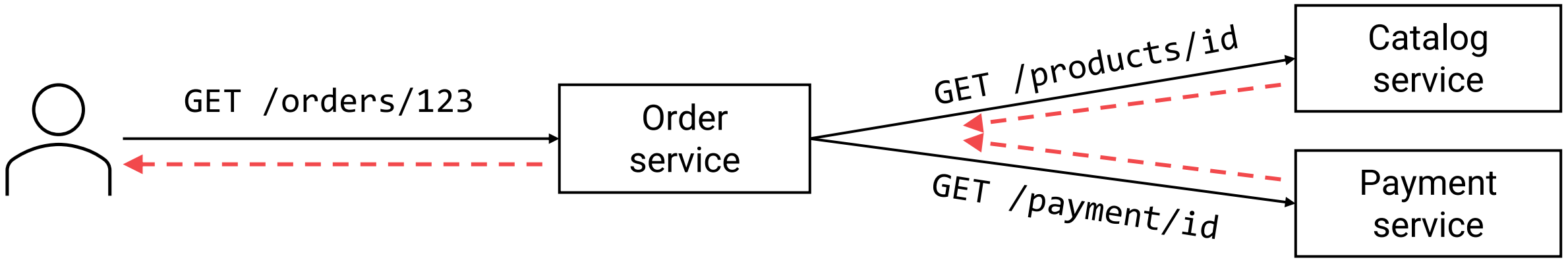
- Asynchronous:
  - The client doesn’t block, and the response, if any, isn’t necessarily sent immediately.
  - The Messaging Model

RESTFUL API

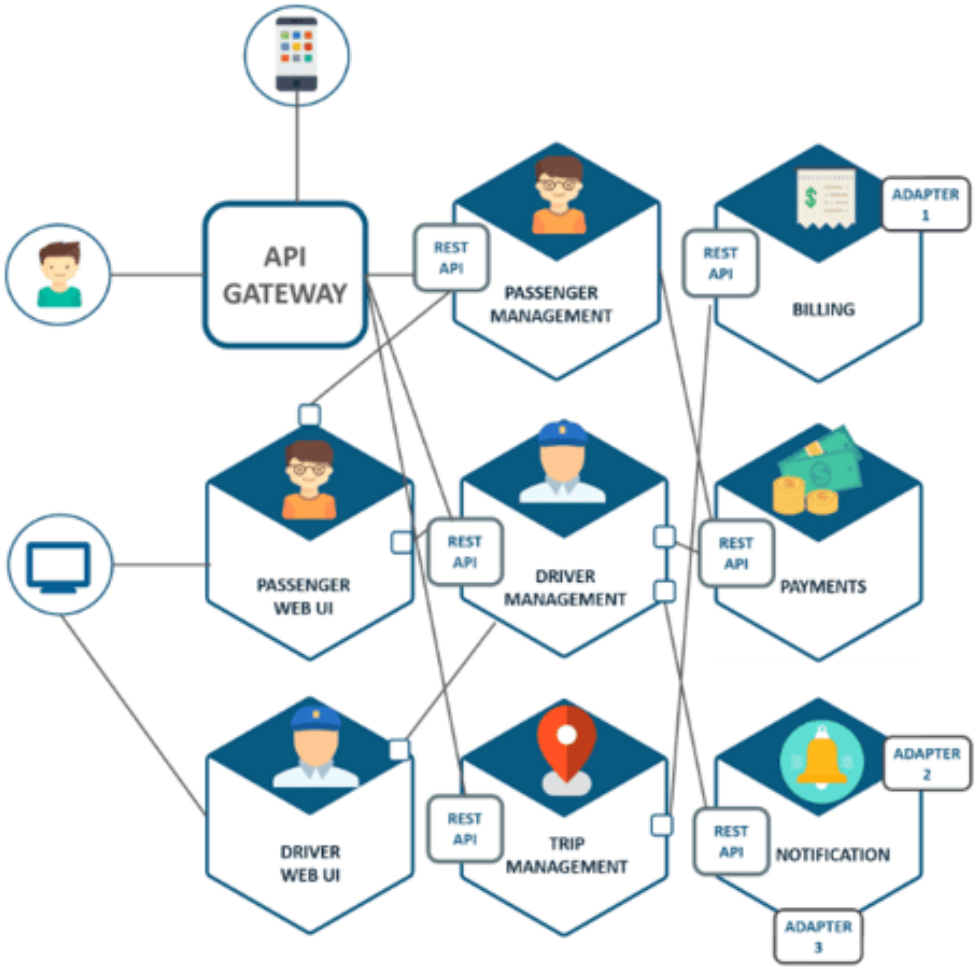
HTTP verb: GET POST PUT DELETE



RESTFUL API



Stateless + Synchronous



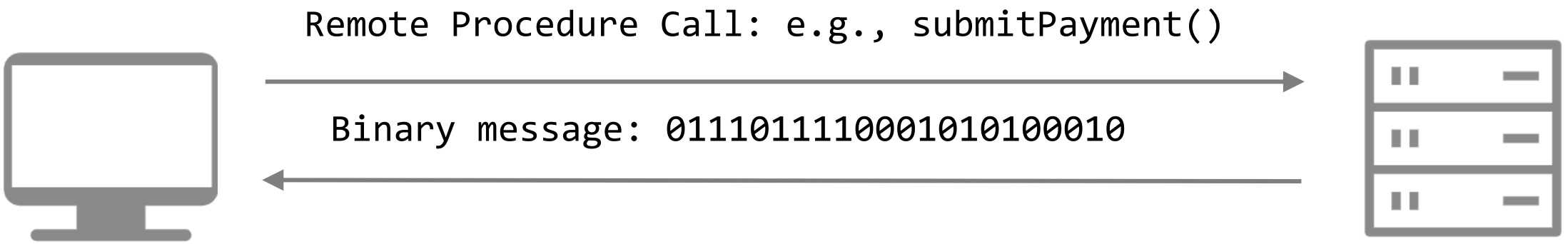
Uber's microservice architecture

https://dzone.com/articles/microservice-architecture-learn-build-and-deploy-a

RESTFUL API

- REST API is not suitable for fetching multiple resources in a single request
- Alternatives
  - GraphQL: Meta (Facebook)
- JSON is text-heavy, could be slow
- Alternatives
  - gRPC: Google, Netflix

gRPC

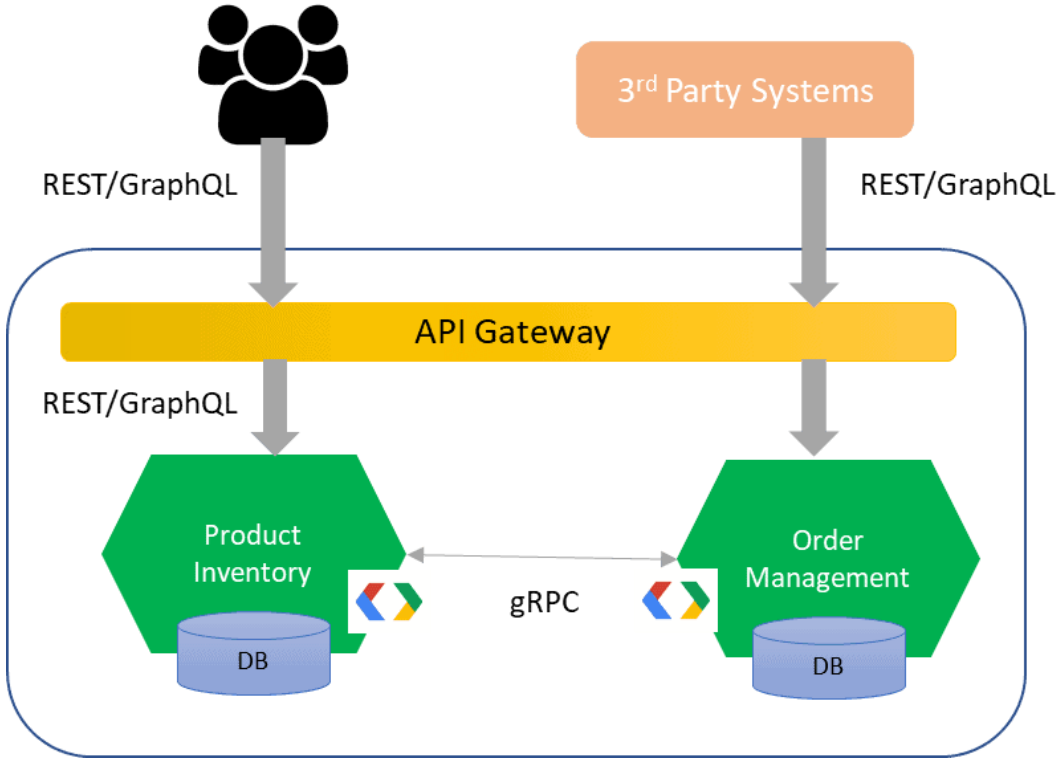


Support more operations (verbs) than REST

Best suited for high-performance or data-heavy microservice architectures.

MIXED COMMUNICATION

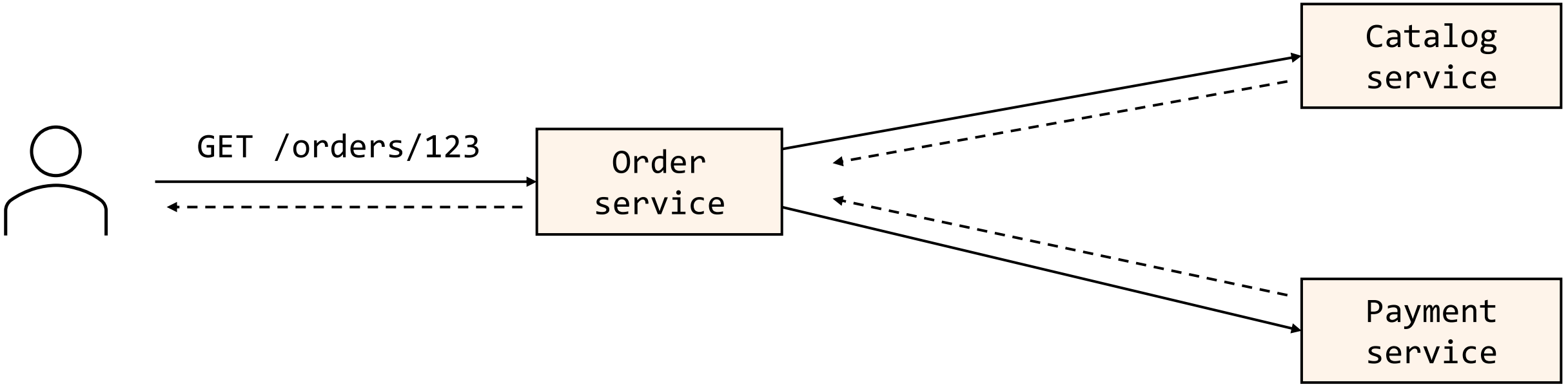
- Client-facing service
  - REST
- Inter-service communication
  - gRPC



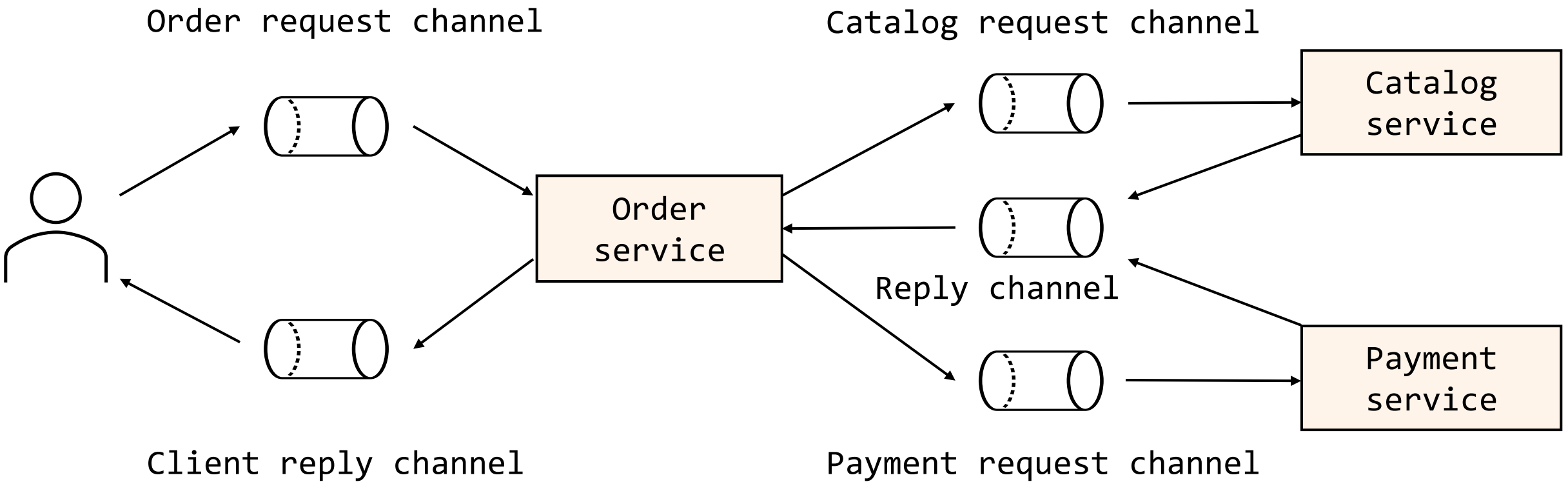
MESSAGING

- Services communicate via messages instead of direct calls
- Built on event-driven architecture principles
- Uses a message broker (e.g., Apache Kafka, RabbitMQ) for publish–subscribe
- Promotes loose coupling and asynchronous processing

REST VS. MESSAGING



REST VS. MESSAGING

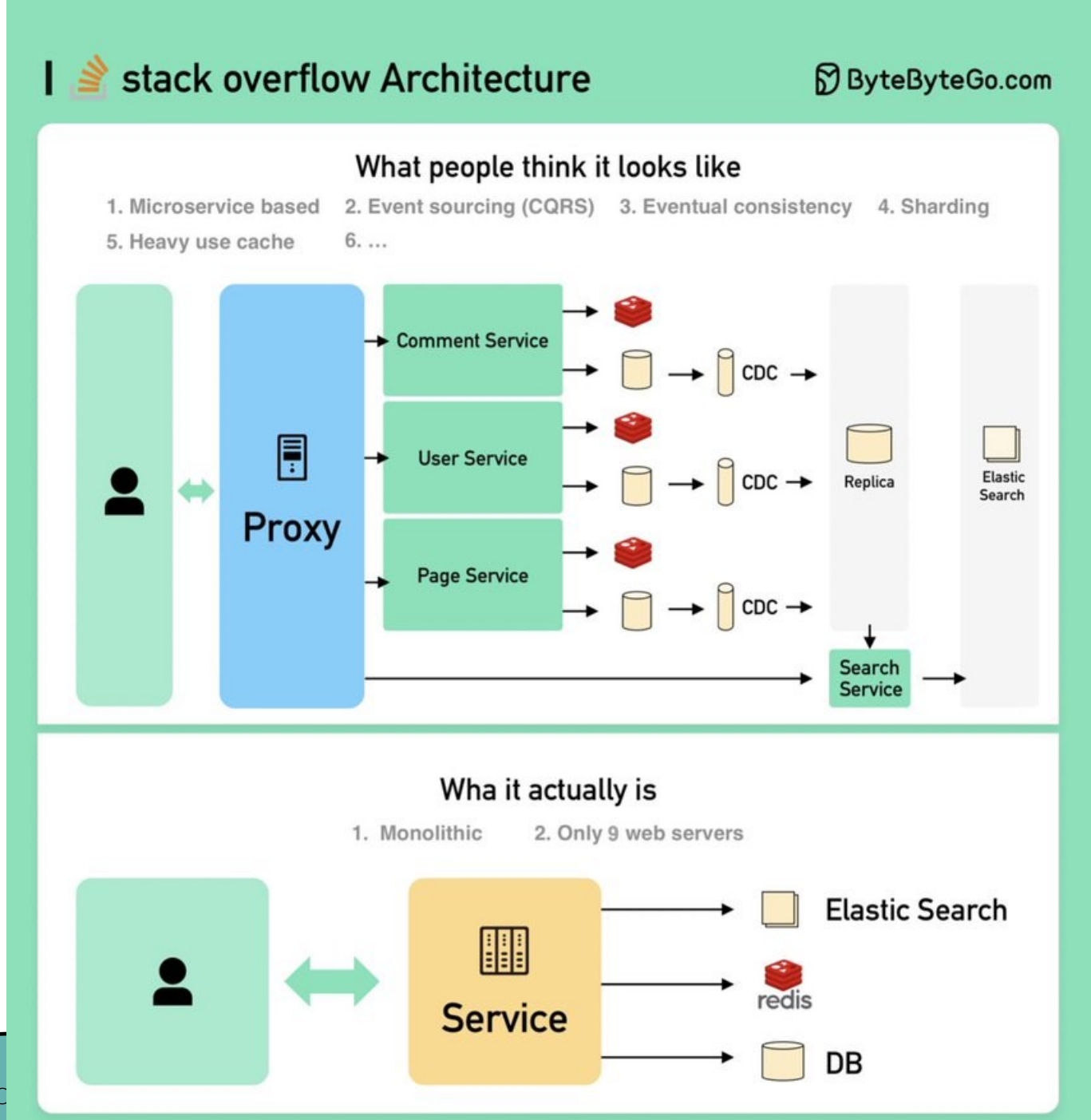


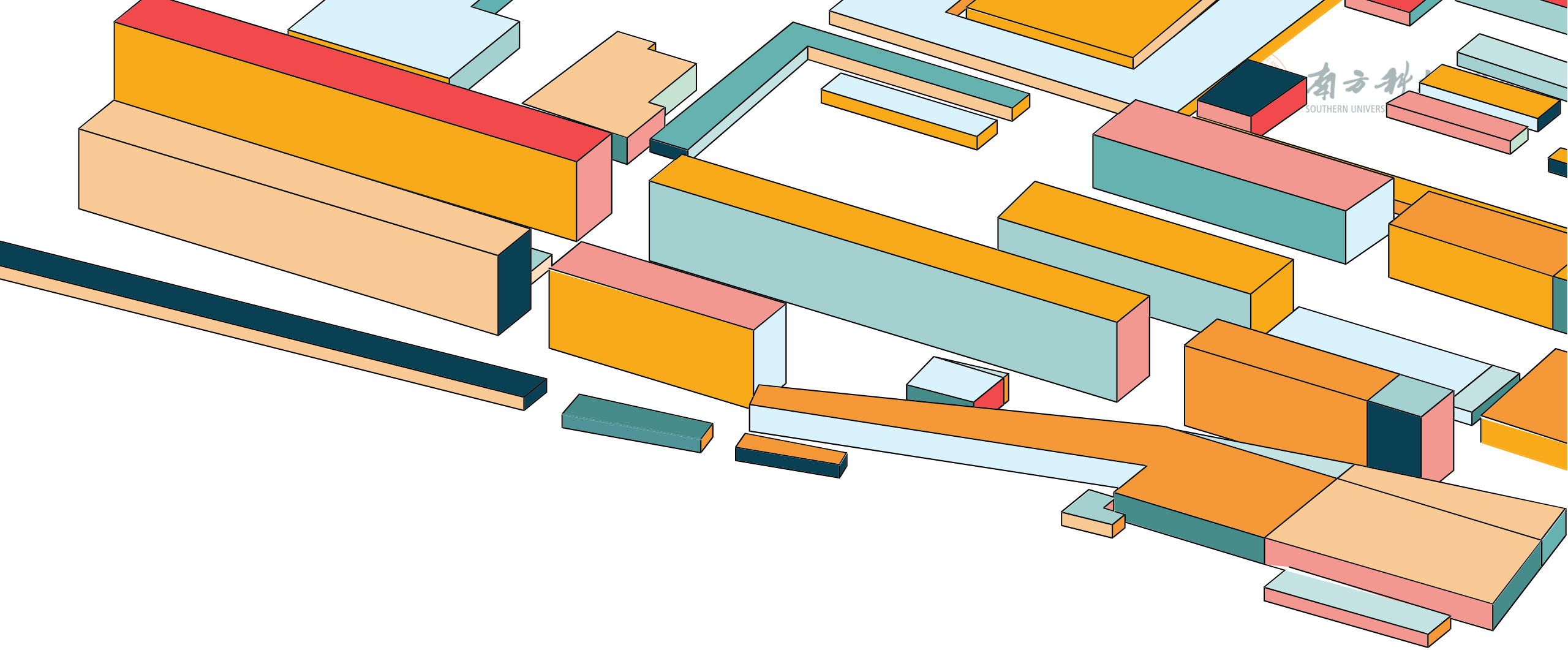
CHOOSING ARCHITECTURAL STYLES

- We've introduced only a subset of available architectural styles
- Once requirements engineering uncovers the characteristics and constraints of the system to be built, the architectural style that best fits those characteristics and constraints can be chosen.
- Different architectural styles are NOT mutually exclusive; instead, they are often applied in combination (e.g., a layered style can be combined with a data-centered architecture in many database applications.)

CHOOSING ARCHITECTURES

- No good or bad architecture (We are not saying microservice is better than monolithic)
- Choose what's suitable based on your application, resources, user base, business style, team structure, etc.
- Simple is often complicated enough

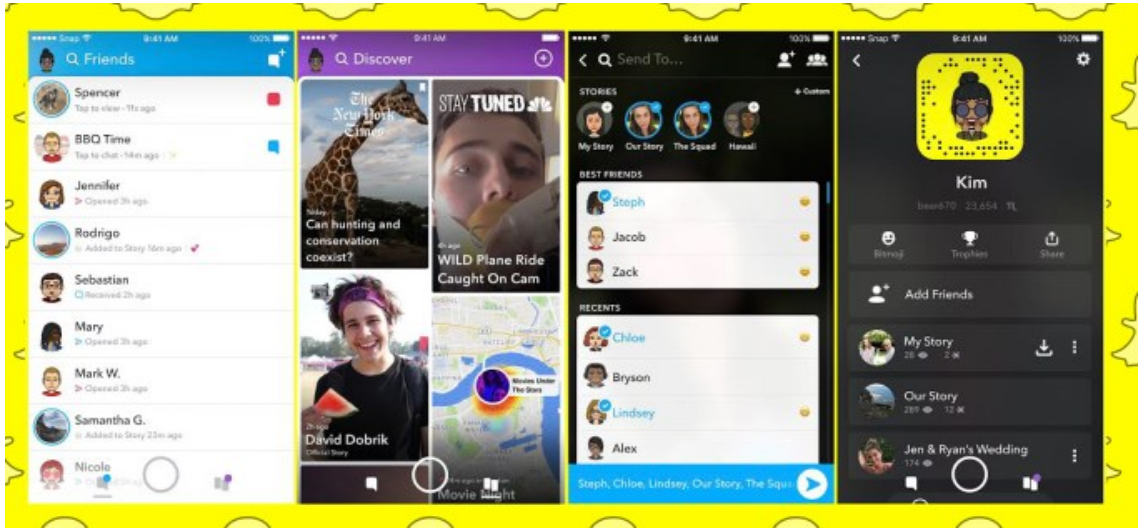


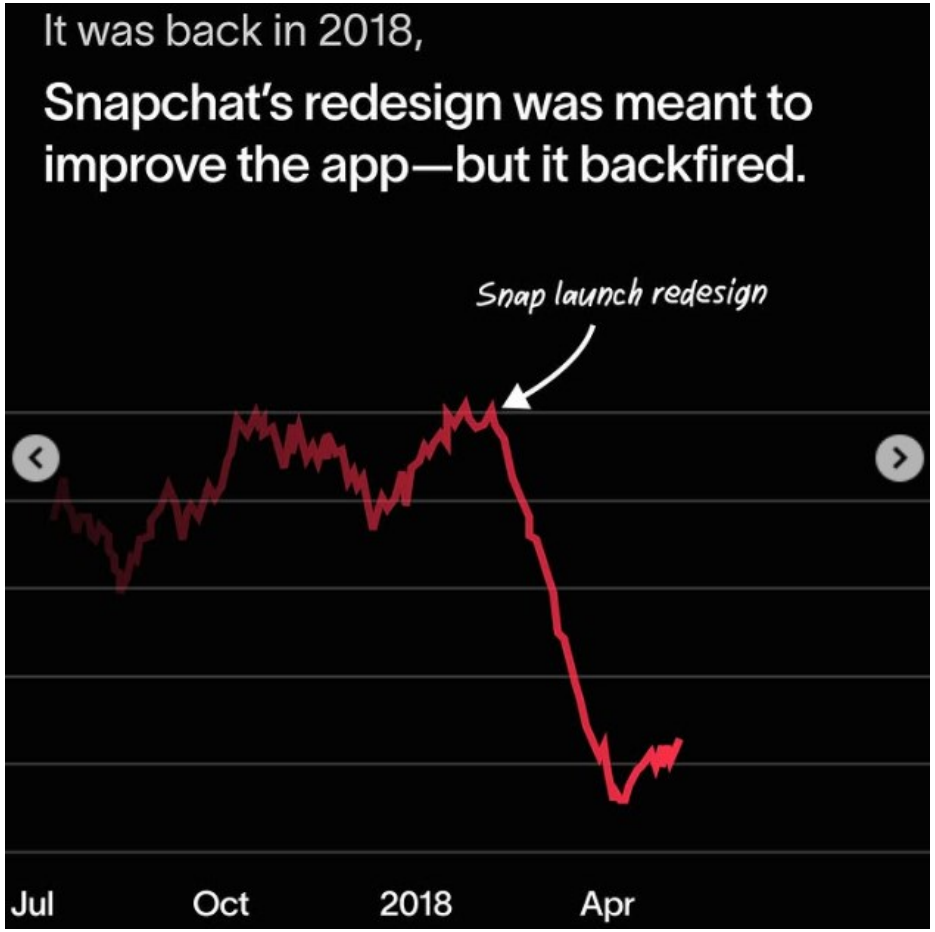


SOFTWARE UI DESIGN

UI DESIGN FAILURE

In 2018, Snapchat's UI redesign faced significant backlash and was widely considered a failure.

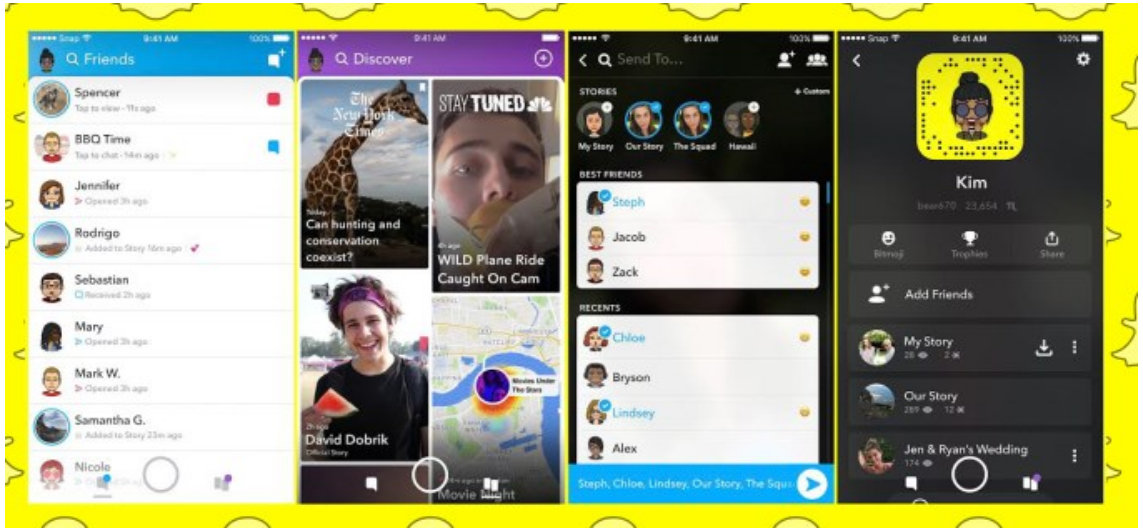


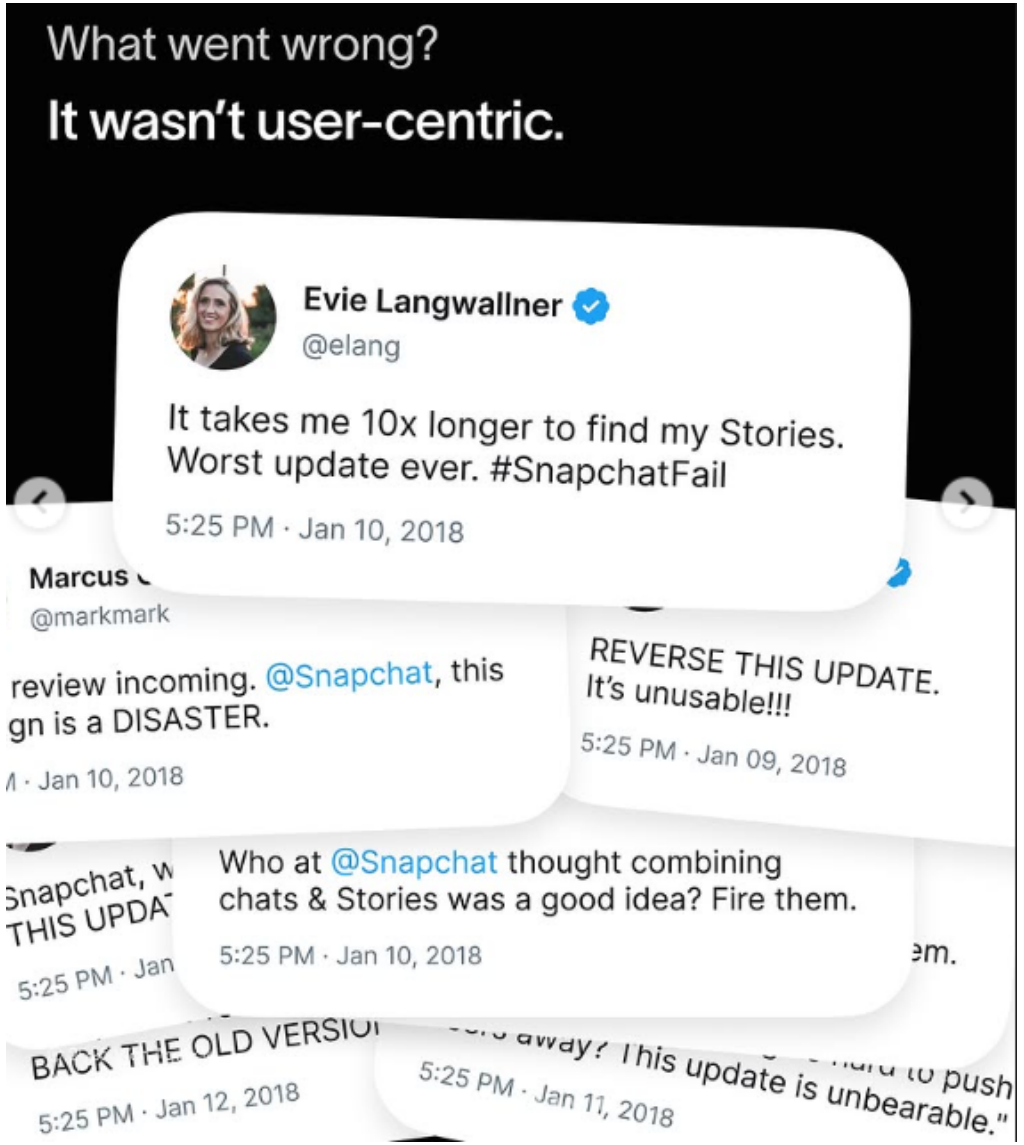


Source: https://www.instagram.com/won.agency/p/DEzluNrJK6B/?img_index=2

UI DESIGN FAILURE

Over 1.2 million users signed a petition requesting the company to revert the design changes





Source: https://www.instagram.com/won.agency/p/DEzluNrJK6B/?img_index=2

UI DESIGN

Visual Design

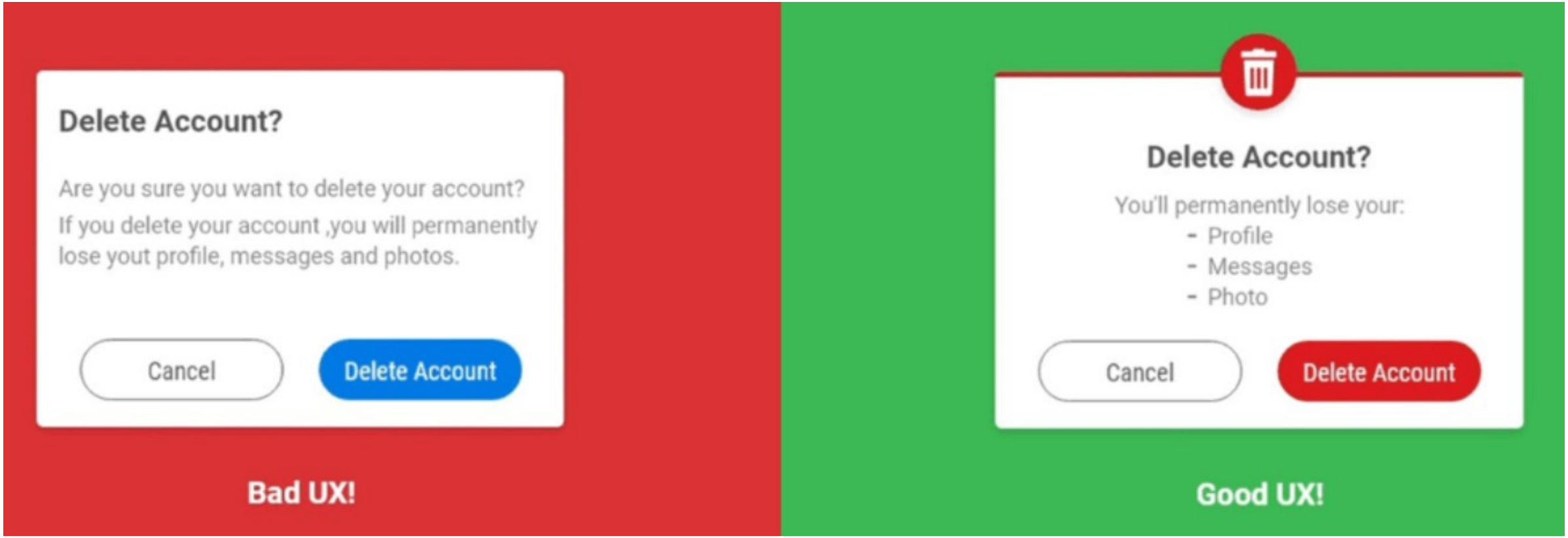
- Focuses on aesthetics, layout, color, typography, and visual hierarchy.
- Enhance usability and user experience.

Interaction Design

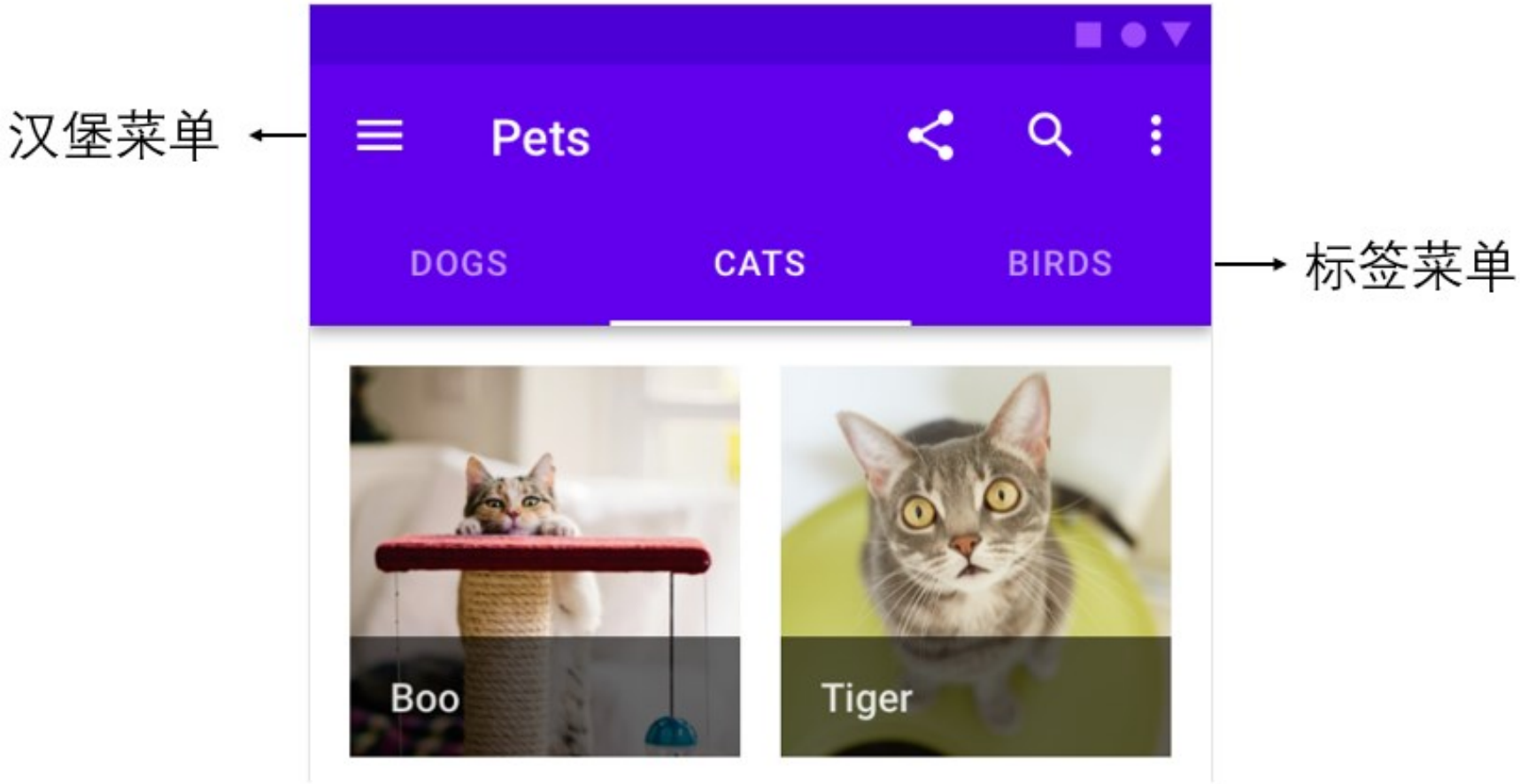
- How users interact with UI elements (buttons, forms, navigation).
- Goal: Minimize friction and cognitive load.

CONSISTENCY

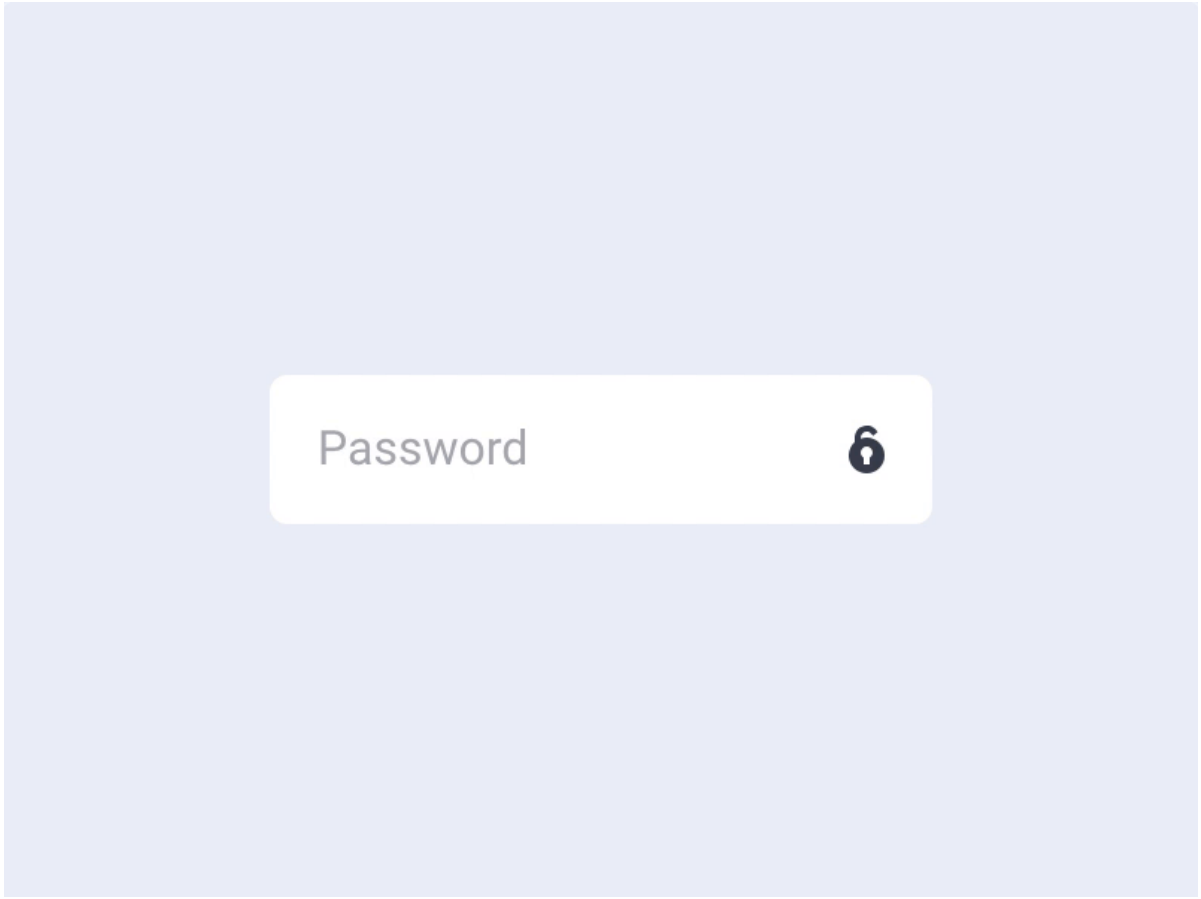
CLARITY


Delete Account?
Are you sure you want to delete your account?
If you delete your account ,you will permanently lose yout profile, messages and photos.
Cancel
Delete Account
Bad UX!
Delete Account?
You'll permanently lose your:
- Profile
- Messages
- Photo
Cancel
Delete Account
Good UX!

ACCESSIBILITY



FEEDBACK

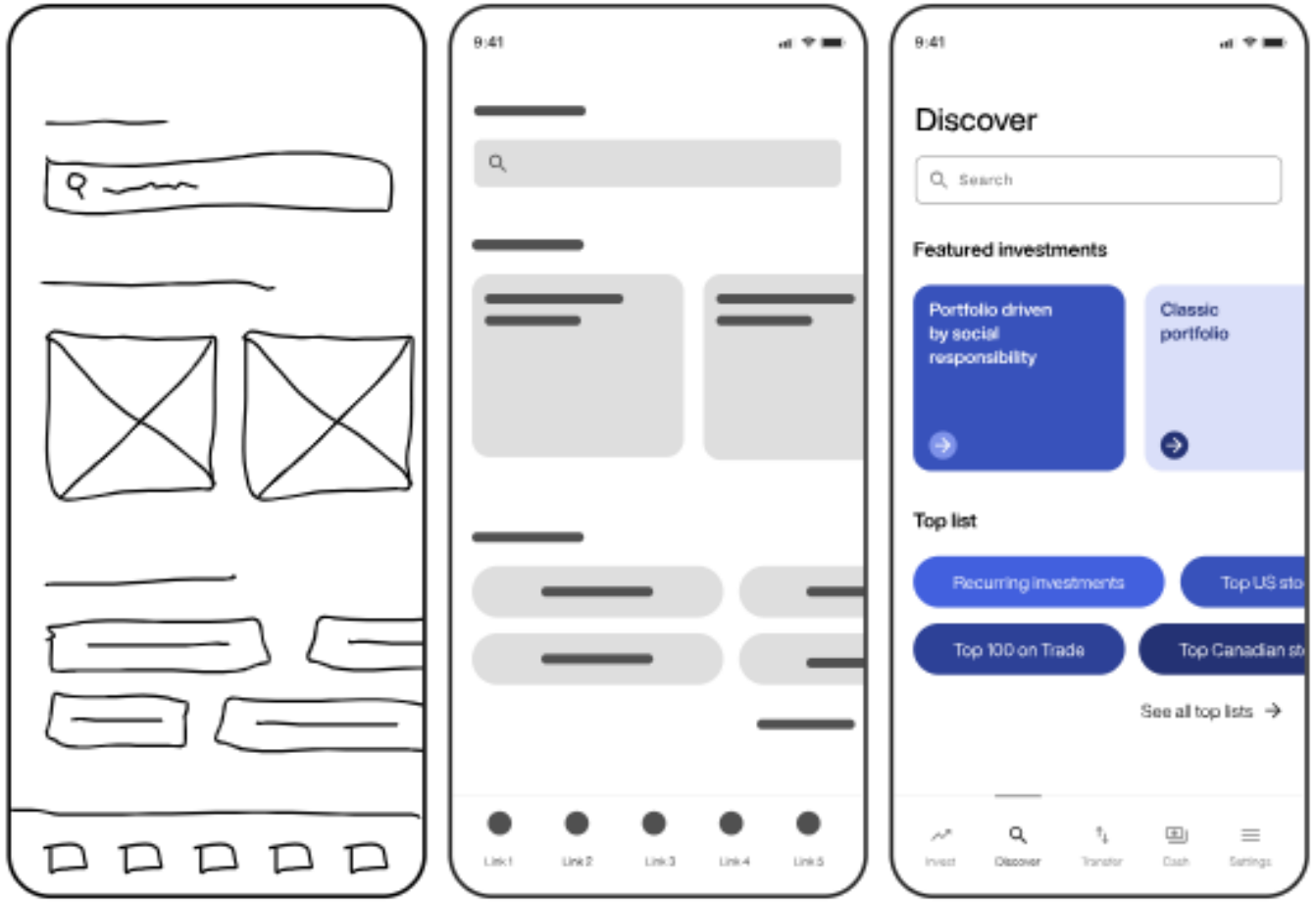


CASE STUDY: HARMONYOS DESIGN

https://developer.huawei.com/consumer/cn/design/devstart/

PROTOTYPING

- A prototype is an early sample or model of a user interface used to test concepts and gather feedback.
- Helps visualize design, test usability, and refine interactions before development.

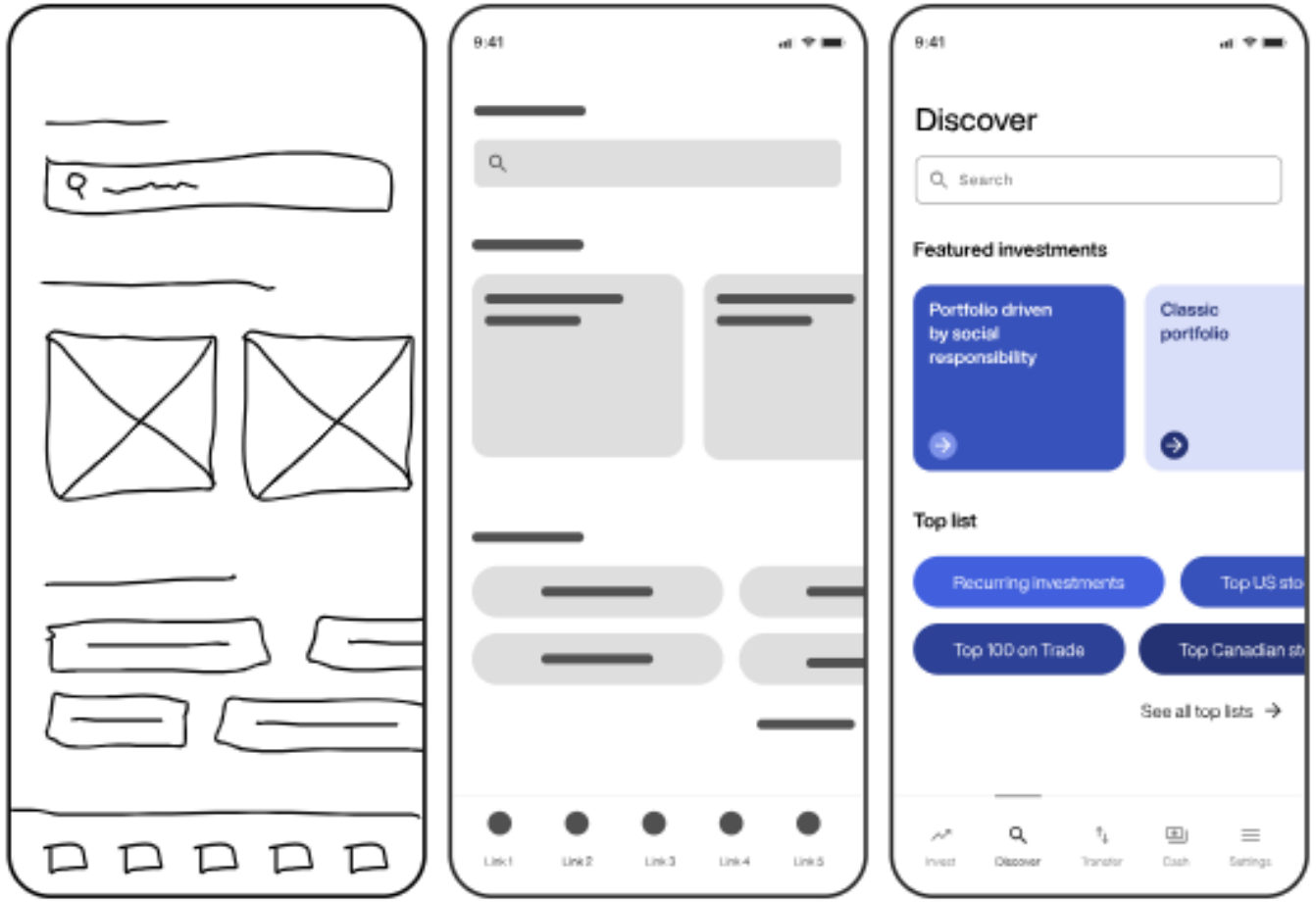


PROTOTYPING

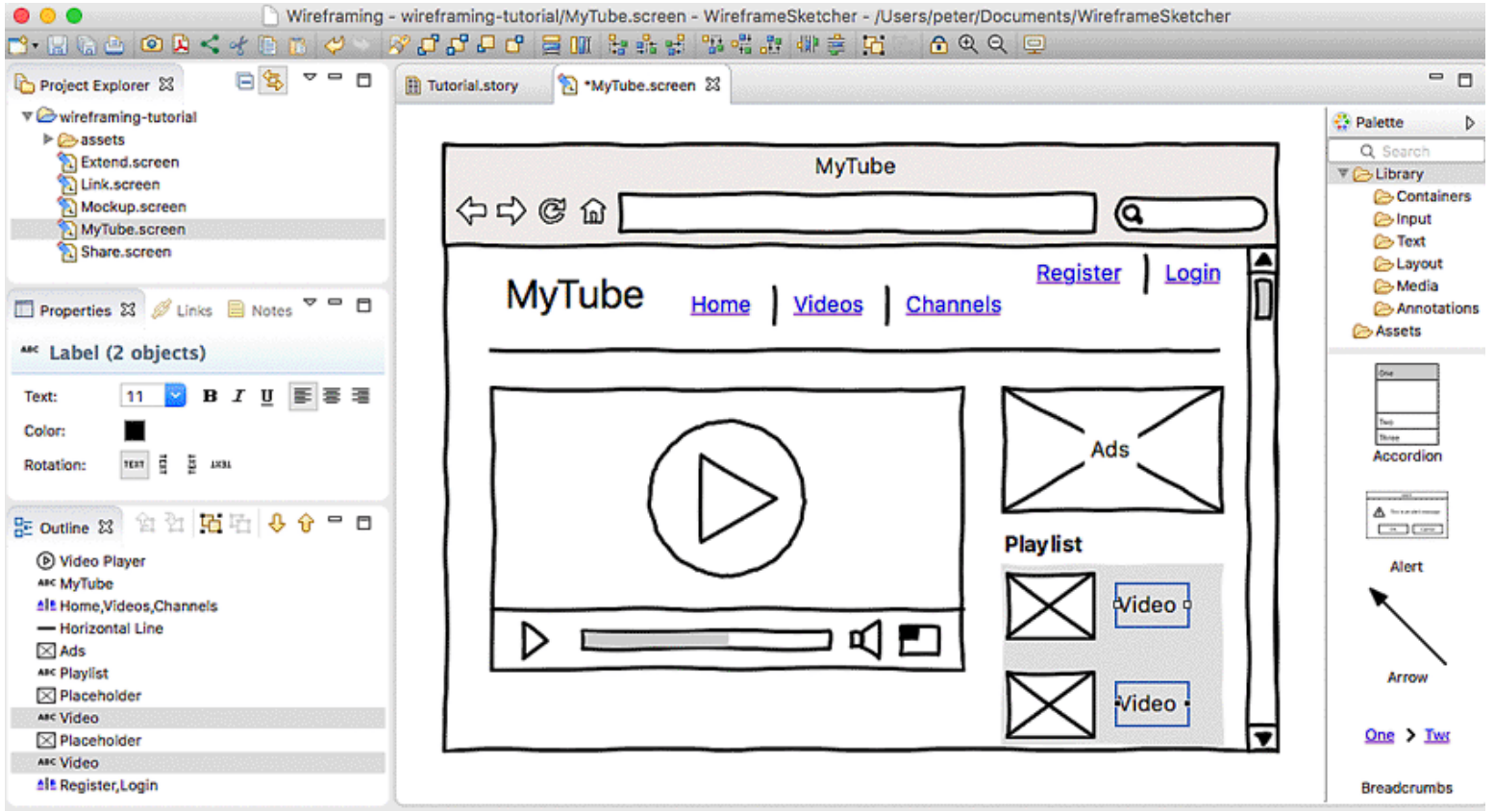
Low-Fidelity (Lo-Fi) Prototypes (低保真)
- Hand-drawn sketches or wireframes.
- Quick and cost-effective.

Mid-Fidelity Prototypes (中保真)
- Digital wireframes with basic interactions.
- Used for layout and navigation testing.

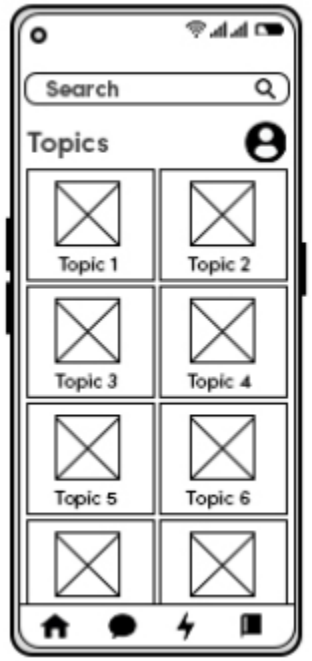
High-Fidelity (Hi-Fi) Prototypes (高保真)
- Interactive and visually detailed.
- Simulates final product experience.

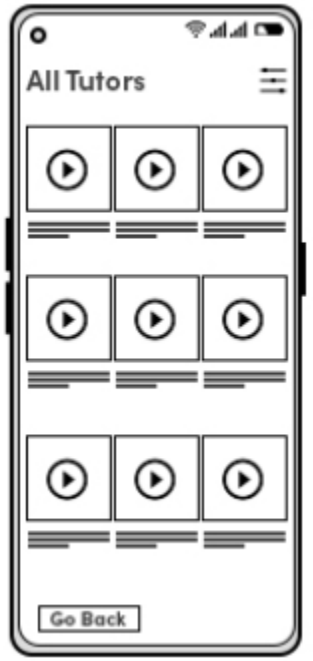


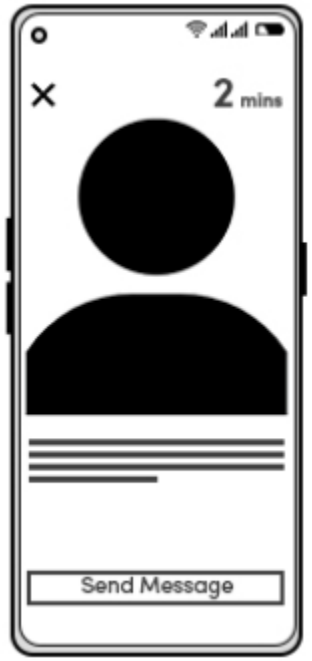
WIREFRAMING TOOLS



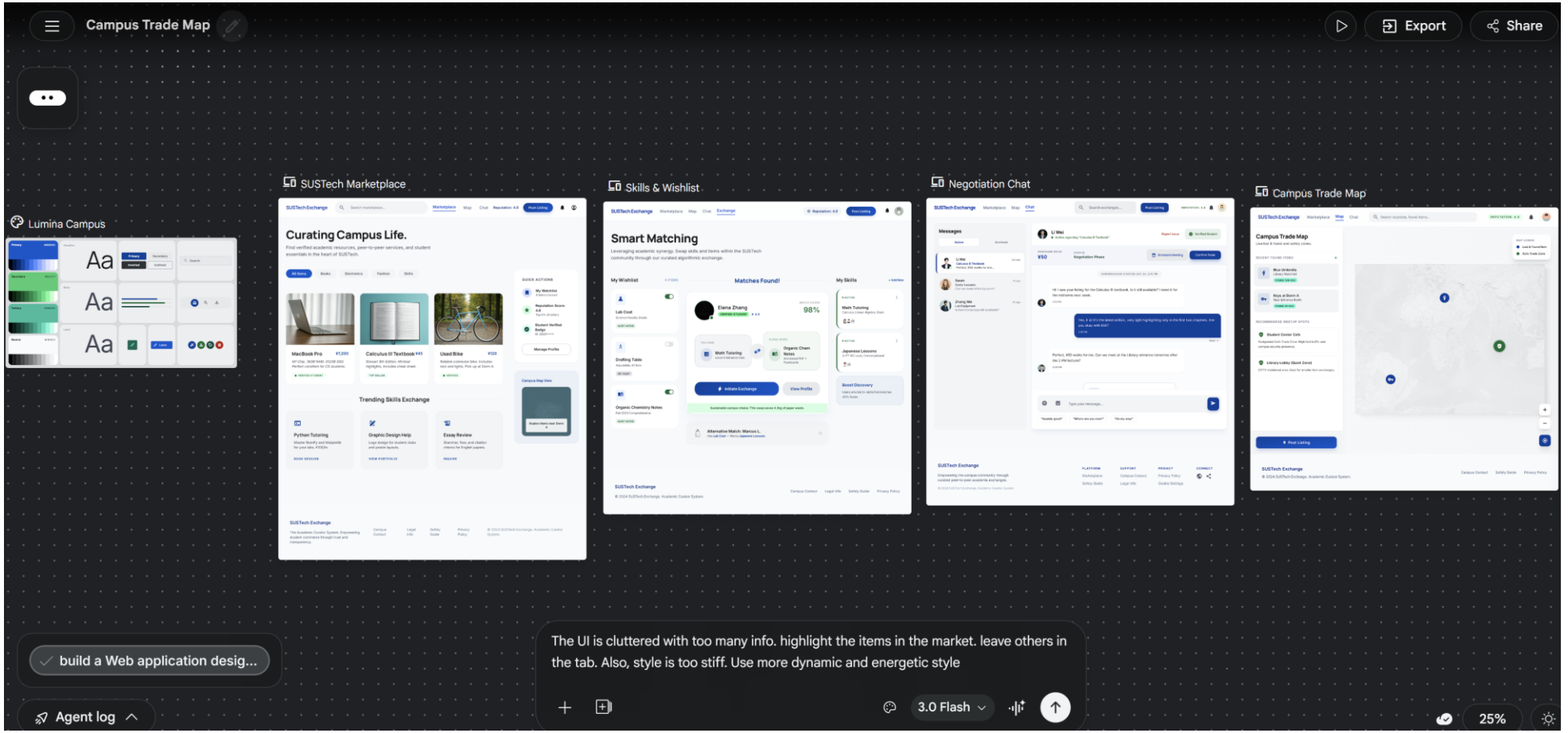
WIREFRAMING TOOLS





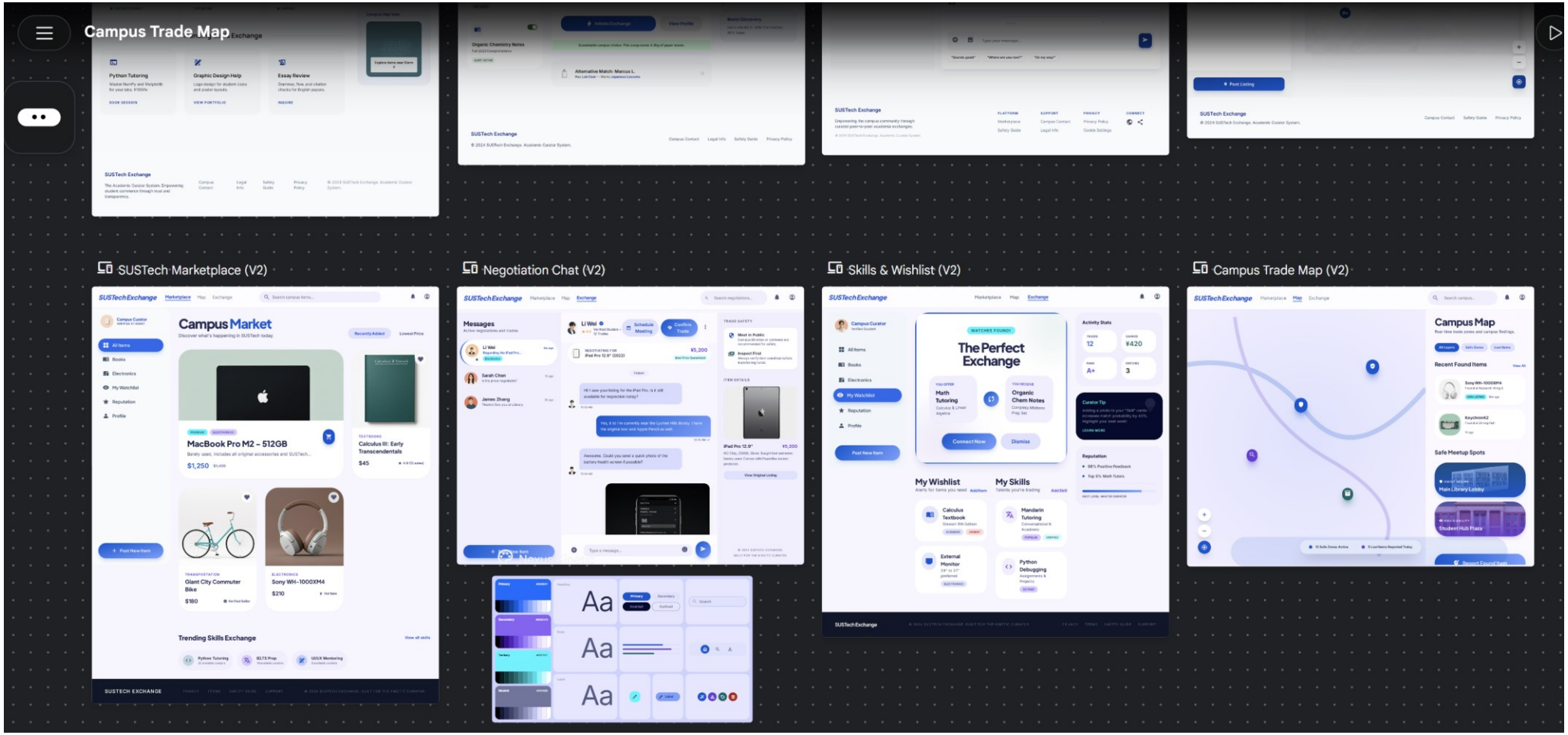


AI TOOLS FOR UI DESIGN



https://stitch.withgoogle.com/

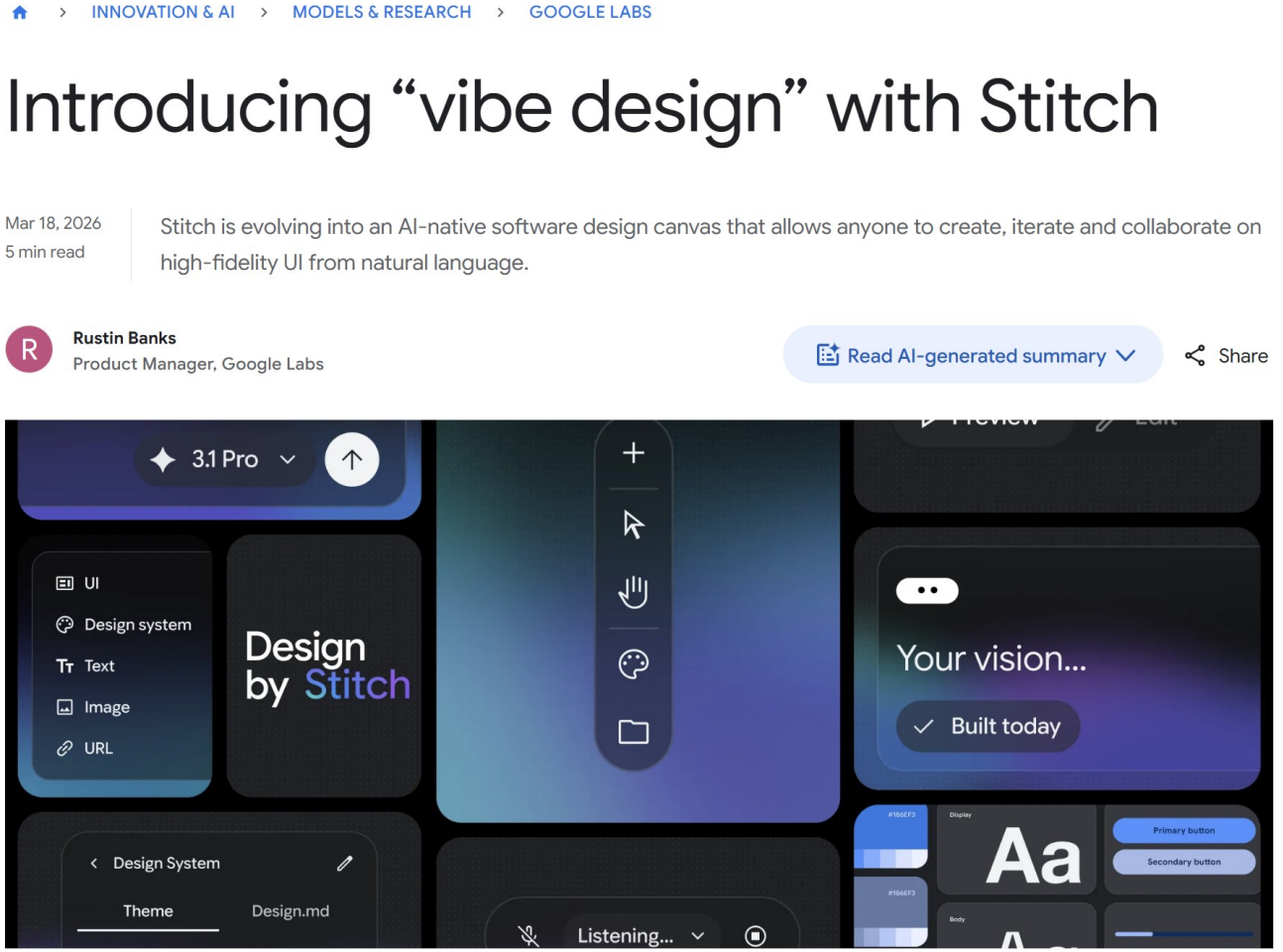
AI TOOLS FOR UI DESIGN



https://stitch.withgoogle.com/

READINGS

https://blog.google/innovation-and-ai/models-and-research/google-labs/stitch-ai-ui-design/

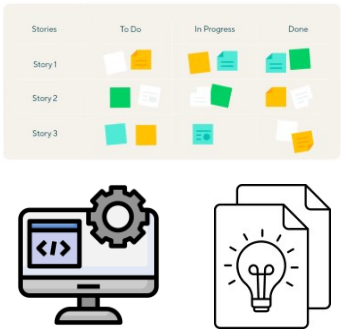


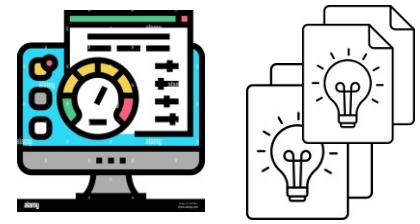
READINGS


超级畅销书
《深入理解 Java 虚拟机》作者又一新作
国内多位架构专家联袂推荐
FENIX
ARCHITECTURE
BUILDING A RELIABLE LARGE-SCALE
DISTRIBUTED SYSTEM
周志明 著
凤凰
架构
构建可靠的
大型分布式系统
从架构演进、架构设计思维、分布式基石、
不可变基础设施、技术方法论 5 个维度全面
探索如何构建可靠的大型分布式系统
机械工业出版社
China Machine Press

OUR TEAM PROJECT

1. Architectural design
2. UI design
3. Git collaboration
4. Demo
5. Scrum board for the next sprint





Week 1
Team up

Week 5
Proposal

Week 9
Sprint 1

Week 16
Sprint 2

NEXT

- Build Systems